LeafGuard: An On-Device Plant Disease Detection System with Explainable AI

IEEE-Format Project Report

Kushagra Dubey

Roll No.: 2300321530109

B.Tech, Department of Computer Science & Engineering

ABES Engineering College, Ghaziabad, Uttar Pradesh, India

kushagra.23b1531207@abes.ac.in

November 6, 2026

A project report on the design, training, and on-device deployment of a domain-adapted self-supervised plant disease classifier with explainable AI.

Abstract

This report documents the complete design, implementation, and evaluation of LeafGuard, a production Android application for on-device plant disease detection that diagnoses 38 leaf diseases across 14 crop species and explains every prediction with an occlusion-sensitivity saliency map. The system is the endpoint of a deliberate, seven-iteration development process in which field top-1 accuracy rose from 11% (a PlantVillage-overfit Swin-Tiny model) through 49%, 34%, and 67.3% to a final 86.7%, each version a direct response to the documented failure mode of its predecessor. The shipped classifier is a self-supervised DINOv2 ViT-S/14 backbone with a 38-way linear disease head and a gradient-reversal domain-adversarial head, fine-tuned in three progressively unfrozen stages that combine CutMix/MixUp, consistency regularization, three-fold field oversampling, and a 3-view test-time augmentation. The PyTorch checkpoint is converted to an 86.8 MB FP32 TensorFlow Lite graph in which ImageNet normalization and the final softmax are baked in, verified bit-identical to the ONNX reference (argmax agreement 3/3, maximum absolute difference 1×10^{-6}), and guarded at runtime by a SHA-256 model-integrity check. The report covers the motivation and literature, the layered Android architecture (MVVM with Clean Architecture, Hilt dependency injection, Room persistence, CameraX capture, PIN authentication), the dataset construction, the full model-and-training methodology including the version-by-version evolution, the PyTorch→ONNX→TFLite conversion pipeline, the on-device occlusion-sensitivity explainability path, the experimental results and confusion analysis, the implementation details of the inference and security subsystems, the limitations, and the ranked improvement roadmap toward 90%+ field accuracy.

Index Terms— Plant disease detection, self-supervised learning, DINOv2, vision transformer, domain adaptation, gradient reversal layer, on-device inference, TensorFlow Lite, explainable AI, occlusion sensitivity, Android, model integrity, agriculture.

Contents

I Introduction 1
I-A Background and Motivation . . . 1
I-B Project Objectives . . . 1
I-C Scope and Contributions . . . 1
I-D Report Organization . . . 1

II Related Work 1
II-A Plant Disease Classification . . . 1
II-B The Lab-to-Field Domain Gap . . . 2
II-C Self-Supervised Features for Agriculture . . . 2
II-D On-Device Deployment and Explainability . . . 2

III System Architecture 2
III-A Architectural Overview . . . 2
III-B Navigation Graph . . . 3
III-C Threading and Concurrency . . . 3
III-D Memory Budget . . . 3
III-E Security Model . . . 3
III-F Dependency Injection . . . 3

IV User Interface and Screen Design 4
IV-A Design Language . . . 4
IV-B Authentication Flow . . . 4
IV-C Dashboard . . . 4
IV-D Plants and Plant Detail . . . 5
IV-E Scan and Result . . . 5
IV-F History, Records, and Images . . . 5
IV-G Accessibility and Theming . . . 6

V Dataset Construction 6
V-A Source Pool . . . 6
V-B Deduplication and Integrity . . . 6
V-C Class Mapping . . . 6
V-D Splits . . . 6

VI Model and Training Methodology 7
VI-A Backbone Choice . . . 7
VI-B Dual-Head Architecture . . . 7
VI-C Progressive Unfreezing . . . 7
VI-D Augmentation and Regularization . . . 7
VI-E Test-Time Augmentation . . . 8
VI-F Version-by-Version Evolution . . . 8

VII Conversion and On-Device Deployment 10
VII-A Conversion Pipeline . . . 10
VII-B Normalization and Softmax Baking . . . 10
VII-C Domain Head Removal . . . 10
VII-D Bit-Identical Verification . . . 10
VII-E Runtime Model Integrity . . . 10

VIII Explainable AI: Occlusion Sensitivity 10
VIII-A Why Occlusion Sensitivity . . . 10
VIII-B The Occlusion Sweep . . . 11
VIII-C Smoothing and Colour Mapping . . . 11
VIII-D Streaming and the Stale-Heatmap Guard . . . 11

IX Experimental Results 11
IX-A Experimental Setup . . . 11
IX-B Headline Accuracy . . . 12
IX-C Per-Class Behavior . . . 12
IX-D Field Confusion Analysis . . . 13
IX-E Bit-Identical Conversion Verification . . . 13
IX-F On-Device Latency and Footprint . . . 13

X Implementation Details 13
X-A DiseaseClassifier . . . 13
X-B SaliencyGenerator . . . 14
X-C ModelIntegrity . . . 14
X-D Room Persistence and Migrations . . . 14
X-E Records Export . . . 14
X-F PIN Authentication . . . 14
X-G Test Coverage . . . 14

XI Testing, Continuous Integration, and Release Engineering 14
XI-A Test Strategy . . . 14
XI-B JVM Unit Test Inventory . . . 15
XI-C Instrumented Tests . . . 15
XI-D Continuous Integration . . . 15
XI-E Build Configuration . . . 15
XI-F Model Asset in CI . . . 15
XI-G Release Cadence and Backup Strategy . . . 15

XII Ablation Study 16

XIII Dataset Exploratory Analysis 16
XIII-A Class Composition . . . 16
XIII-B Domain Composition . . . 16
XIII-C Field Validation Set Characteristics . . . 17

XIV Training Dynamics 17
XIV-A Loss Trajectory . . . 17
XIV-B Why Progressive Unfreezing Matters . . . 18

XV Detailed Code Listings 18
XV-A DiseaseClassifier: 3-View TTA . . . 18
XV-B SaliencyGenerator: Occlusion Sweep . . . 18
XV-C ModelIntegrity: SHA-256 Guard . . . 19
XV-D ScanDao: Room Persistence . . . 19
XV-E PinHasher: PBKDF2 Derivation . . . 19
XV-F LeafGuardNavHost: Auth Gating . . . 19

XVI Suggestion and Advisory Engine 19
XVI-A From Label to Action . . . 19
XVI-B Severity Tiers and Staged Treatment . . . 20
XVI-C Knowledge-Base Authoring . . . 20

XVII Comparative Evaluation 20
XVII-A Position Relative to the Literature . . . 20
XVII-B What the Comparison Does and Does Not Show . . . 20
XVII-C Feature Comparison . . . 20

XVIII Additional Implementation Listings 20
XVIII-A SuggestionEngine: Label to Advisory . . . 20
XVIII-B CsvGenerator: RFC 4180 Export . . . 21
XVIII-C LeafGuardDatabase: Migrations . . . 21
XVIII-D InferenceModule: Hilt Binding . . . 22

XIX Per-Version Hyperparameter Reference 22

XX Future System Architecture 22
XX-A INT8 Quantized Variant . . . 22
XX-B Active Learning Loop . . . 22
XX-C DINOv2-Base and On-Device Grad-CAM . . . 22
XX-D Ensemble of DINOv2-Small and ConvNeXt-Tiny . . . 22

XXI Additional Code Listings (Application Layer) 22
XXI-A ScanViewModel: Capture to Classification . . . 22
XXI-B AuthRepositoryImpl: State Transitions . . . 23
XXI-C LeafGuardApplication: Hilt Entry . . . 23
XXI-D PdfExporter: On-Device PDF Records . . . 23

XXII Discussion 23
XXII-A Why the Field Gap Closed . . . 23
XXII-B Limitations . . . 24
XXII-C Threats to Validity . . . 24

XXIII Improvement Roadmap 24
XXIII-A Field Data and Hard Mining (Highest Leverage) . . . 24
XXIII-B INT8 Quantization . . . 24
XXIII-C Per-ABI APK Splits and Packaging . . . 24
XXIII-D Larger Backbone and On-Device Grad-CAM . . . 24
XXIII-E Low-Yield Directions . . . 24

XXIV Conclusion 25

Appendix 25
A Installation . . . 27
B First Launch and Registration . . . 27
C Scanning a Leaf . . . 27
D Interpreting the Heatmap and the Integrity Banner . . . 27
E Records and Sharing . . . 27
F Notebook 1: leafguard-finetune-v1 (mobile baseline) . . . 27
G Notebook 2: leafguard-swin-finetune-v1 (multi-source PV pool) . . . 27
H Notebook 3: leafguard-swin-finetune-v2 (first field data) . . . 28
I Notebook 4: leafguard-swin-finetune-v2-1 (cap enforcement) . . . 28
J Notebook 5: leafguard-swin-finetune-v2-2 (three-stage schedule) . . . 28
K Notebook 6: leafguard-swin-finetune-v3 (ImageNet pivot) . . . 28
L Notebook 7: leafguard-swin-finetune-v3-5-0-0 (DINOv2 + DANN, final) . . . 28
M Training Environment . . . 29
N Re-running a Version . . . 29
O Application Build . . . 29

References 29

List of Figures

1 Layered architecture of the LeafGuard Android application . . . 4
2 The LeafGuard navigation graph . . . 5
3 Dataset composition by source, gathered versus retained . . . 6
4 Fine-tune architecture . . . 8
5 The three-phase progressive-unfreeze schedule . . . 8
6 Field and mixed top-1 accuracy across the seven development . . . 9
7 The PyTorch->ONNX->TFLite conversion pipeline . . . 11
8 The occlusion-sensitivity pipeline . . . 12
9 Field, lab, and mixed top-1 accuracy . . . 12
10 Precision and recall for selected strongest and weakest . . . 13
11 Top field confusion pairs on the 525-image field validation . . . 13
12 Estimated additive contribution of each v3->v3.5 component . . . 17
13 Per-class validation support on the mixed split, across the . . . 17
14 Schematic training-loss trajectory across the three phases . . . 18
15 The suggestion engine maps a predicted label to a severity . . . 20
16 Indicative field or cross-domain accuracy of representative . . . 21

List of Tables

I Data sources pooled for fine-tuning (raw counts before dedup). 6
II Three-phase progressive-unfreeze schedule. 7
III Version-by-version evolution 9
IV Top-1 accuracy (%, with 3-view TTA) and footprint. 12
V Weakest and representative strongest classes (mixed split). 12
VI Estimated per-component field-accuracy contribution (v3→v3.5). 16
VII Feature comparison of LeafGuard against representative related systems. 21
VIII Per-version training hyperparameters (Swin/DINOv2 line). 23
IX The 38 PlantVillage classes (canonical index order). 26
X v3.5 (DINOv2 + DANN) training hyperparameters. 26
XI GitHub release history of the LeafGuard application. 27
XII Requirements and acceptance matrix. 28

I. Introduction

A. Background and Motivation

Plant diseases are a persistent and economically significant threat to global food security. Independent assessments estimate that between 10% and 40% of potential crop yield is destroyed each year by foliar pathogens, with the heaviest losses falling on the smallholder farmers who produce the majority of food in many regions [1], [2]. A smallholder who notices a lesion on a leaf typically has no access to a plant pathologist and must decide, with little information, whether to apply a chemical treatment, remove the plant, or wait. A smartphone camera that can identify the disease at the moment it is spotted, estimate its severity, and recommend a staged treatment is therefore a directly useful tool, and the past decade has produced a large body of deep-learning classifiers that approach or exceed human accuracy on benchmark datasets [3]–[5].

The canonical benchmark, PlantVillage, is a laboratory collection of single leaves photographed against a uniform grey background under controlled lighting [2]. On this distribution, modern convolutional and transformer-based classifiers routinely report 98% or higher validation accuracy. The difficulty is that this benchmark distribution is very different from the photographs a grower actually captures in a field: real field photographs show whole plants, cluttered backgrounds, variable illumination, overlapping leaves, and lesions at varying angles and scales. The gap between the two distributions is large enough that a model reporting 98% on PlantVillage routinely collapses to roughly 30–40% on real field photographs [6], [17]. This lab-to-field domain gap is the first major obstacle that keeps high-accuracy classifiers out of growers' hands.

The second obstacle is deployment and trust. Most published pipelines either depend on a remote server—which is unavailable to a smallholder without connectivity, and which raises privacy concerns about transmitting farm photographs—or expose no explanation for a prediction. A grower who is told to spray a chemical or remove a plant on the basis of a single confidence score has no way to judge whether the model's attention was on the lesion or on an irrelevant patch of background. Recent mobile efforts [6], [8] and explainable-AI analyses [9], [10], [20] address parts of this gap, but a single deployed system that is simultaneously field-robust, fully offline, and explainable has remained elusive.

B. Project Objectives

The LeafGuard project was undertaken to close both gaps in a single production system. The specific objectives were: (1) to ship an Android application that diagnoses 38 PlantVillage leaf-disease classes across 14 crop species entirely on-device, with no network dependency for inference; (2) to make the classifier robust to the lab-to-field domain shift, measured by an explicit field validation set rather than only a lab or mixed set; (3) to attach a model-agnostic explanation to every diagnosis, so the grower can see which regions of the leaf drove the prediction; (4) to make the deployed model auditable through a runtime integrity check; and (5) to document the development process honestly, including the failures of intermediate versions, so the final result is reproducible and the design decisions are auditable.

C. Scope and Contributions

The system reported here, shipped as build v3.5.0.0, meets all five objectives. Its contributions are: a domain-adapted self-supervised backbone (DINOv2 ViT-S/14 [12] with a gradient-reversal domain-adversarial head [13]) that lifts field accuracy from 67.3% to 86.7% over an ImageNet-pretrained Swin-Tiny baseline; a three-phase progressive-unfreeze recipe that preserves the self-supervised pre-training while attacking the domain gap with CutMix/MixUp, consistency regularization, and three-fold field oversampling; a bit-identical PyTorch→ONNX→TFLite conversion pipeline in which normalization and softmax are baked into the graph; a model-agnostic occlusion-sensitivity explainability path that works under the TFLite interpreter because it needs only the forward pass; a SHA-256 runtime model-integrity guard; and a documented seven-iteration development arc in which field accuracy rose from 11% to 86.7%.

D. Report Organization

The remainder of this report is organized as follows. Section II surveys related work in plant disease classification, the lab-to-field domain gap, self-supervised features for agriculture, and on-device explainability. Section III describes the layered Android architecture. Section V details the dataset construction and splits. Section VI presents the model and the full training methodology, including the version-by-version evolution. Section VII describes the conversion and on-device deployment pipeline. Section VIII details the occlusion-sensitivity explainability path. Section IX reports the experimental results and confusion analysis. Section X covers the implementation of the inference and security subsystems. Section XXII discusses why the field gap closed and the limitations. Section XXIII gives the ranked improvement roadmap, and Section XXIV concludes.

II. Related Work

A. Plant Disease Classification

Early deep-learning plant disease classifiers were convolutional networks trained on PlantVillage [1], [2]. Mohanty, Hughes, and Salathé demonstrated in 2016 that a fine-tuned AlexNet/GoogLeNet could reach 99.35% accuracy on the 38-class PlantVillage benchmark, establishing both the feasibility and the benchmark-saturation problem that still motivates this work. Subsequent work established that vision transformers [14] and their hierarchical variants

such as the Swin Transformer [15] match or exceed convolutional accuracy on the lab benchmark while offering better global context modelling.

Within the last two years the literature has diversified considerably. Murugavalli and Gopi [3] proposed a vision-transformer pipeline for precision agriculture reporting 99.8%+ training and validation accuracy and 99.83% external-test accuracy, and Nishankar et al. [4] showed that ConvNeXt-Small and Swin-Small variants are effective for robust tomato leaf disease recognition. Arshad et al. [5] introduced ConvGeM-next, an efficient Swin-based framework with generalized mean pooling for plant disease detection. These works report very high accuracy, but they are evaluated predominantly on the lab distribution and say little about real field imagery. The high lab numbers, while technically impressive, do not transfer to field photographs, which is precisely the gap LeafGuard targets.

B. The Lab-to-Field Domain Gap

The lab-to-field gap is now a recognized research problem in its own right rather than a footnote to benchmark papers. Sharif et al. [6] explicitly observe that models trained on PlantVillage perform poorly on real field images and propose a federated multimodal edge-learning method to improve generalization while keeping data on-device. Salman, Muhammad, and Han [17] report the gap quantitatively: a plain Vision Transformer, trained on a balanced PlantVillage subset and tested on the in-the-wild PlantDoc dataset, drops sharply in accuracy, and they recover roughly 20 percentage points of the lost accuracy with a mixture-of-experts ViT, reaching 68% cross-domain accuracy. That 68% figure is a useful reference point for the field accuracy LeafGuard ultimately achieves.

Jeon et al. [7] tackle the same gap with unsupervised domain adaptation between lab and field domains. Prasad and Agniraj [16] combine the Domain-Adversarial Neural Network (DANN) framework [13] with CORAL for cross-domain plant disease detection, and Islam et al. [19] recently proposed a domain-adapted lightweight MobileNet ensemble for few-shot plant disease classification that stays near 40 MB for mobile deployment. Our work is closest in spirit to these domain-adaptation approaches but differs in three respects. First, we use a self-supervised (DINOv2) backbone rather than an ImageNet-supervised one, on the reasoning that self-supervised patch features are less over-specialized to ImageNet object categories and therefore more stable under the PlantVillage-to-field shift. Second, we pair domain adaptation with a progressive-unfreeze schedule that preserves the self-supervised pre-training rather than letting the adversarial signal catastrophically forget it. Third, we report results on an explicit, held-out field validation split rather than only on a mixed or augmented test set, so the field number is an honest measurement rather than a side effect of test-time augmentation on a lab-dominated set.

C. Self-Supervised Features for Agriculture

DINOv2 [12] learns general-purpose visual features without labels, trained on a 142-million-image self-supervised objective, and is markedly more robust to distribution shift than ImageNet-supervised pre-training. Rithiga and Nadesh [11] recently integrated self-supervised learning with vision transformers for an explainable plant disease framework, motivating the use of self-supervised backbones in this domain. Bai et al. [18] built DINOV2-FCS on the DINOv2-B backbone for joint fruit-leaf disease classification and severity segmentation, reaching 99.67% classification accuracy and demonstrating strong cross-disease generalization—evidence that the self-supervised representation transfers well within plant pathology. These results motivated our choice to build directly on DINOv2 ViT-S/14 and to quantify the field-accuracy benefit of the self-supervised backbone over an ImageNet-Swin baseline.

D. On-Device Deployment and Explainability

Mobile plant disease applications increasingly target on-device inference via TensorFlow Lite to avoid the connectivity and privacy problems of server-backed pipelines [6], [8]. For explainability, Grad-CAM and its variants are the dominant gradient-based methods [10], [20], and Alhammad et al. [20] provide a recent example that integrates Grad-CAM into a VGG16-based potato disease classifier to visualize the regions driving each prediction. However, Natarajan, Chakrabarti, and Margala [9] compare gradient-based and occlusion-based explainable-AI techniques for plant diseases and find occlusion methods to be a robust, model-agnostic alternative that does not depend on access to internal model state.

This distinction is consequential for on-device deployment. The TensorFlow Lite interpreter, which is the standard on-device inference engine on Android, exposes neither intermediate layer activations nor gradients to the application layer. True Grad-CAM is therefore unavailable on-device without a custom delegate or operator tap, which is a substantial engineering effort. Occlusion sensitivity, by contrast, requires only the forward pass: each region of the image is masked in turn and the change in the predicted class confidence is recorded. Because the deployed classifier is already wrapped behind a single forward-pass interface, occlusion sensitivity can be implemented as a loop over that interface with no model internals. LeafGuard adopts occlusion sensitivity for exactly this reason, and Section VIII details its on-device implementation, including the smoothing and colour-mapping that turn the raw per-cell confidence drops into a clinically legible heatmap.

III. System Architecture

A. Architectural Overview

LeafGuard is a native Android application written in Kotlin and built with the Jetpack Compose UI toolkit. It follows a layered MVVM with Clean Architecture, in

which a pure-Kotlin domain layer isolates the business rules from both the Android framework and the UI, a data layer wraps all framework concerns (Room persistence, the TensorFlow Lite interpreter, encrypted PIN storage, CameraX capture), and a Compose UI layer renders state exposed by Hilt-injected ViewModels. The layering is enforced by dependency direction: the UI layer depends on the domain layer's interfaces, the domain layer depends on no Android types, and the data layer implements the domain interfaces and is bound to them by Hilt at compile time. This separation is what allows the inference, persistence, and explainability subsystems to be developed and unit-tested independently of the UI.

Fig. 1 shows the four layers and their responsibilities. The design is conventional for a production Android application, but three aspects deserve emphasis because they are specific to an on-device ML application. First, the TFLite Interpreter is a singleton bound in a Hilt InferenceModule and loaded into a direct ByteBuffer via readBytes(), which is compression-safe and avoids the memory pressure of streaming a large model. Second, all Room, TFLite, and CameraX work runs on Dispatchers.IO, with a dedicated single-thread dispatcher for the TFLite interpreter because the interpreter is not thread-safe. Third, the occlusion-sensitivity second pass shares the same interpreter as the headline classification, so a single model asset serves both the fast diagnosis path and the slower explanation path.

B. Navigation Graph

The application's navigation is organized as a single NavHost that gates authentication against the main application. The start destination is the registration screen on first launch, then the login screen on subsequent launches, and then a bottom-navigation graph over five destinations (dashboard, plants, scan, history, records). An auth-state StateFlow is observed in MainActivity and routes the user to register, login, or dashboard from AuthState (Unregistered, Locked, Unlocked). The gating effect is carefully scoped: it only force-navigates while the user is on an auth route, so in-app navigation is never hijacked once the user is inside the main graph. From the dashboard the user can reach the plants list, a per-plant detail screen, the scan screen (optionally prefilled with a plant id), and the result screen for a given scan identifier. A separate images gallery groups image-bearing scans by plant. The full graph, including the auth-gated entry, is shown in Fig. 2.

C. Threading and Concurrency

All persistence, inference, and camera work runs off the main thread. Room DAOs use Flow return types for observable queries and suspend functions for mutations, executed on an @IoDispatcher qualifier-bound dispatcher. ViewModels expose StateFlow<UiState>, and the UI collects with collectAsStateWithLifecycle so state observation is tied to the lifecycle and released when the screen leaves the composition. Inference is wrapped in a suspending withContext(Dispatchers.Default) call so the UI never blocks. The TFLite interpreter deserves special attention because it is not thread-safe: two concurrent interpreter.run calls crash the native runtime. The regression this design prevents is that a user starts a scan, leaves it (the fast-path classify completes and the 64-pass occlusion heatmap keeps running on an application-scoped coroutine), then starts a second scan from the scan page; without serialization the second scan's fast-path classify would collide with the first scan's mid-flight heatmap. The classifier therefore serializes the single forward pass through a synchronized(interpreter) lock held only for the duration of one run, so a waiting classify interleaves between the heatmap's forward passes and waits at most one pass.

D. Memory Budget

The application targets devices with 6 GB or more of RAM. The dominant resident footprint is the TFLite interpreter itself: the 86.8 MB DINOv2 v3.5 FP32 model plus the 224×224 activation buffers and the occlusion grid's per-cell re-inference account for roughly 100–160 MB of resident memory. CameraX preview uses ImageAnalysis with STRATEGY_KEEP_ONLY_LATEST and backpressure so only one frame is in flight at a time, dropped if processing is slow. Room cursor windows are paginated via LIMIT/OFFSET in the list DAOs to avoid holding large cursors. Captured scan images are stored as compressed JPEG (quality 85) under context.filesDir/scans/ and never held in memory long-term, so the gallery and history screens read paths rather than bitmaps.

E. Security Model

The application is fully offline: it declares no network permission, makes no network calls, and includes no analytics or advertising SDKs. User authentication is a 6-digit PIN chosen at first launch. The PIN is never persisted in raw form; instead it is fed through PBKDF2-HMAC-SHA256 with 100,000 iterations and a per-install salt, and the derived key is stored in EncryptedSharedPreferences keyed to the Android Keystore. The Keystore-backed master key is hardware-protected on devices that support it. The Room database is not separately encrypted in the current release (file-based encryption is on by default on Android API 26+); SQLCipher can be layered in if a stronger threat model is required. The model asset itself is guarded at runtime by a SHA-256 integrity check detailed in Section VII, and the only outbound network use anywhere in the system is the build-time CI pull of the trained model from a private Kaggle dataset, which happens in CI and never on the phone.

F. Dependency Injection

Hilt generates the dependency graph from the @HiltAndroidApp LeafGuardApplication. Feature modules install

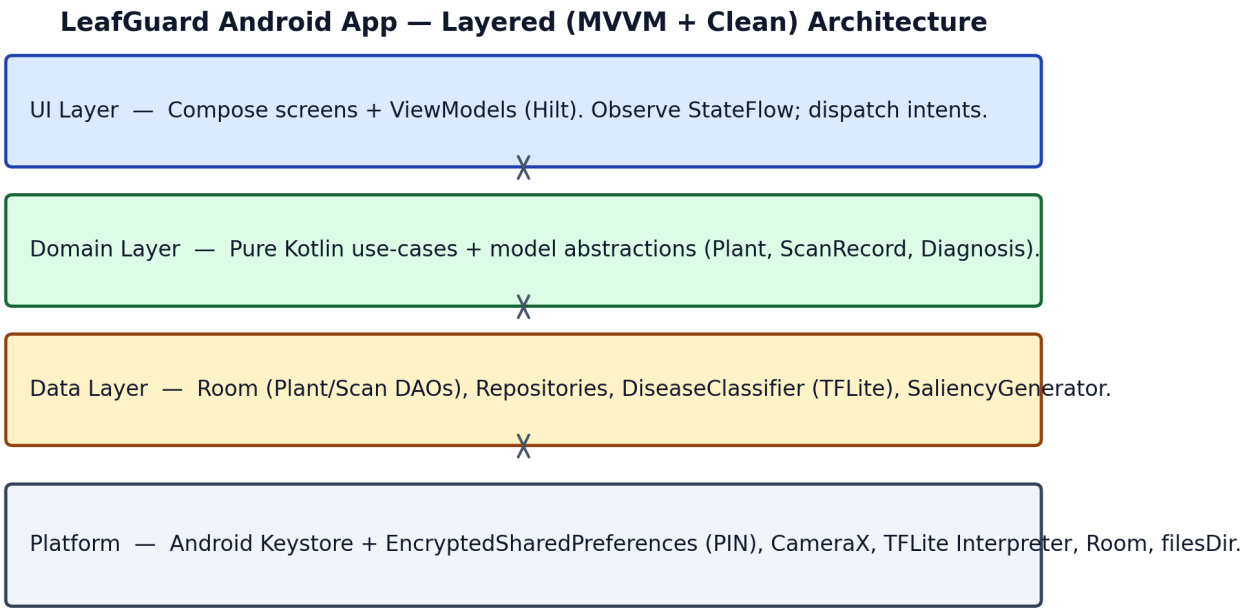


Fig. 1. Layered architecture of the LeafGuard Android application (MVVM with Clean Architecture). The UI layer observes StateFlow from Hilt-injected ViewModels; the domain layer is pure Kotlin with no Android imports; the data layer wraps Room, the TFLite interpreter, encrypted PIN storage, and CameraX.

into SingletonComponent (repositories, the inference engine, the Room database and DAOs) or ViewModelComponent (the ViewModels). KSP processes Hilt and Room in a single pass, which keeps build times reasonable despite the large generated graph. The inference module binds a singleton Interpreter loaded compression-safe, the label and suggestion parsers, the DiseaseClassifier, the SaliencyGenerator, and the SuggestionEngine; this is the module that makes the model swap between the retired Swin asset and the current DINOv2 asset a one-line configuration change plus a SHA-256 digest update, because the I/O contract (raw $[0, 1]$ pixels in, 38-class softmax out) is identical between the two models.

IV. User Interface and Screen Design

A. Design Language

LeafGuard's user interface is built entirely with Jetpack Compose and follows the Material 3 design system with dynamic color support on Android 12+ and a fixed light/dark theme pair on earlier versions. The visual language is deliberately restrained: a single accent colour (a leaf green) is used for primary actions and the active navigation destination, the type scale follows the Material 3 body/heading ramp, and the spacing rhythm is built on a four-density grid. The restraint is intentional, because the screen that matters most—the result screen—must present a diagnosis, a confidence bar, an occlusion heatmap, and a staged treatment list without visual noise that could distract a grower from the actionable information. Every screen is built from a small set of reusable Composables (LeafScaffold for the bottom-nav shell, confidence bars, section headers, advisory cards) so that the visual language is consistent across the nine screens.

B. Authentication Flow

The first-launch experience forces registration: the user chooses a 6-digit PIN, confirms it, and supplies a display name, and only then is the main graph reachable. Returning users are routed to the login screen, where the PIN is entered and validated against the stored derived key. The AuthScreen is wired to the AuthViewModel, which exposes the authState StateFlow and a one-shot failure flow for validation errors (wrong PIN, mismatched confirmation, non-6-digit input). A reset dialog allows clearing the stored credentials and re-registering, which is the recovery path for a forgotten PIN. The gating is implemented in LeafGuardNavHost via an AUTH_ROOT launcher that observes AuthState and routes to register, login, or dashboard; the gating effect is scoped so that it only force-navigates while the user is on an auth route, which prevents the auth gate from hijacking in-app navigation once the user is inside the main graph.

C. Dashboard

The dashboard is the landing screen after authentication and the hub of the bottom navigation. It surfaces live summaries of the scan and plant lists: the total number of tracked plants, the total number of scans, and the most recent diagnosis with its confidence and timestamp. The lists are observed as Flows from the repositories through

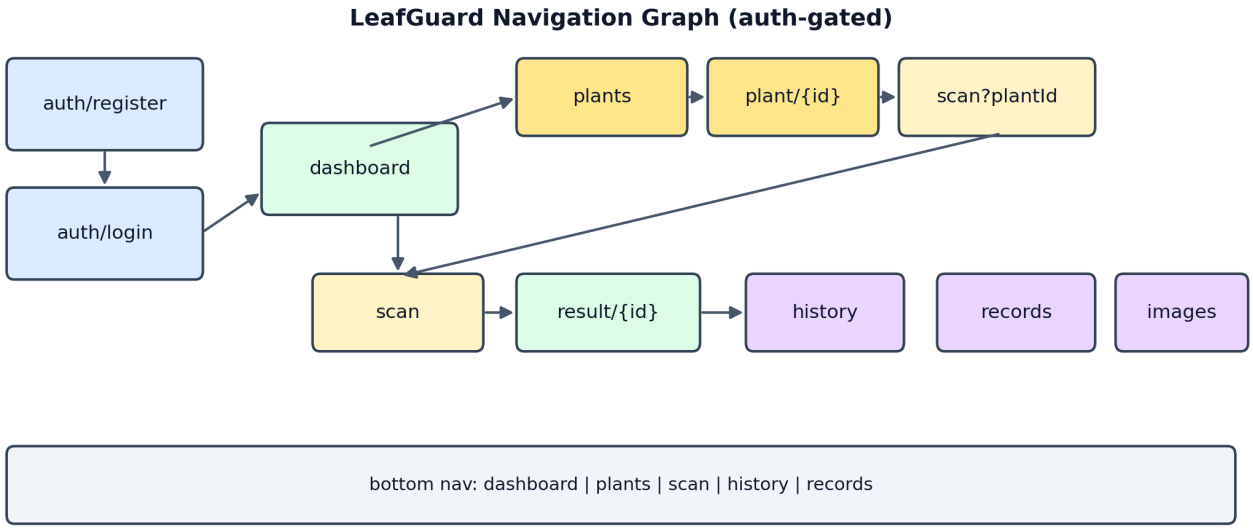


Fig. 2. The LeafGuard navigation graph. Registration and login gate the main graph; the bottom navigation provides five destinations (dashboard, plants, scan, history, records), with the images gallery reachable from the dashboard and the result screen keyed by scan identifier.

the DashboardViewModel, so the dashboard updates in real time as the user adds plants or completes scans elsewhere in the app. Quick-action cards link to the scan screen (start a new scan) and the plants screen (add a plant). The dashboard is intentionally low-density: a grower opening the app to check recent activity should see the salient numbers immediately, not a wall of detail.

D. Plants and Plant Detail

The plants screen lists every tracked plant with its name, species, and a scan count, observed live from the PlantRepository. An add-plant dialog collects name, species, and notes; a per-row delete action removes a plant (with ON DELETE SET NULL on the scan foreign key so the plant's scans are orphaned rather than cascade-deleted). The plant detail screen shows the plant's metadata, an edit mode for name/species/notes, a "Scan this plant" action that navigates to the scan screen with the plant id prefilled, and a delete-with-confirm dialog. The detail screen also surfaces the plant's scan history as a compact list, so a grower can review the disease progression of a single plant over time without leaving its detail page.

E. Scan and Result

The scan screen is the primary interaction. It offers two capture paths: a live CameraX capture (using PreviewView for the viewfinder and ImageCapture for the still) and a photo-picker import for an existing gallery image. On capture or import, the bitmap is passed to the ScanViewModel, which runs the 3-view test-time-augmented classification on Dispatchers.Default, persists the source bitmap to filesDir/scans/ as a compressed JPEG, creates a ScanEntity with the prediction and image path, and navigates to the result screen keyed by the scan id. The result screen renders the predicted condition, a confidence bar for the top label, the source image and the occlusion heatmap side by side once the heatmap completes, the top-k alternative labels, and the staged treatment advisory (immediate, short-term, long-term actions plus a prevention checklist). The heatmap is produced by an application-scoped coroutine that survives the scan screen being popped, so the user sees the diagnosis immediately and the heatmap streams in over the following seconds; a stale-heatmap guard keys each result by scan id so a superseded scan's late heatmap cannot clobber the current result. If the model-integrity check has failed, a warning banner is rendered above the diagnosis rather than blocking the screen.

F. History, Records, and Images

The history screen joins scans to plant names in memory and offers a per-plant filter, producing a HistoryItem list that shows each scan's plant, diagnosis, confidence, and timestamp in reverse chronological order. The records screen groups scans by plant into a per-plant disease log and exports it on-device to CSV (pure RFC 4180, tested) or PDF (Android PdfDocument); this is the feature a grower uses to share a disease history with an extension officer. The images screen groups image-bearing scans (those with a non-null sourceImagePath) by plant and renders them as a Coil AsyncImage gallery, so the grower can visually review every captured leaf and its associated diagnosis without re-scanning. Together, history, records, and images turn a sequence of isolated scans into a longitudinal record per plant, which is the basis for

TABLE I
Data sources pooled for fine-tuning (raw counts before dedup).

Source	Domain	Images
PlantVillage (Augmented)	Lab	175,734
PlantDoc	Field	2,922
field_corn	Field	4,188
field_alexzcheny	Field	160
Total gathered	—	183,004
Kept after dedup	—	90,973

tracking disease progression and treatment efficacy over a season.

G. Accessibility and Theming

The UI observes collectAsStateWithLifecycle so state collection is tied to the screen's lifecycle and released on disposal, which avoids unnecessary recomposition and battery drain. The Material 3 theme provides light and dark variants, with a night theme (values-night/themes.xml) for the splash and system bars. Text sizes follow the Material 3 ramp and meet the minimum touch-target sizes for the primary actions. The bottom navigation uses five destinations (dashboard, plants, scan, history, records) with the scan destination centrally placed as the most-used action; the images gallery is reachable from the dashboard rather than occupying a nav slot, keeping the bottom bar uncluttered. Adaptive launcher icons (mipmap-anydpi-v26) and a leaf-logo foreground drawable give the app a recognizable identity across OEM launcher shapes.

V. Dataset Construction

A. Source Pool

The fine-tune pool deliberately mixes the canonical lab distribution with real field imagery, because a model trained only on PlantVillage learns to exploit the uniform grey background rather than the lesion evidence, and that cue is exactly what fails in the field. Four sources were pooled, totaling 183,004 images before deduplication (Table I). The PlantVillage (Augmented) source provides the lab anchor: 175,734 single-leaf laboratory images against a uniform background, covering all 38 classes. Three field sources provide real-world coverage: PlantDoc (2,922 field photographs across 27 of the classes), a corn-specific field collection (4,188 images over 4 corn classes), and a smaller field-acquired dataset (160 images). The field sources are essential because they are the only data on which the field validation split can be built and on which domain-adaptive training can operate.

B. Deduplication and Integrity

The gathered pool contained a large number of duplicates, both within sources and (in earlier iterations, across sources). Deduplication uses an MD5 hash of the first 16 KB of each file combined with the file size, applied within each source. This within-source, partial-hash strategy was chosen after an earlier cross-source full-file SHA-256 dedup ran for approximately nine hours and timed out on the Kaggle T4 runtime; the within-source MD5-of-prefix strategy removes the vast majority of exact and near-exact duplicates in a fraction of the time. After deduplication, 90,973 images were retained (92,031 duplicates removed). Integrity checking verifies every retained image with PIL.Image.verify() before it enters the training pipeline; zero corrupt images were found in the final pool.

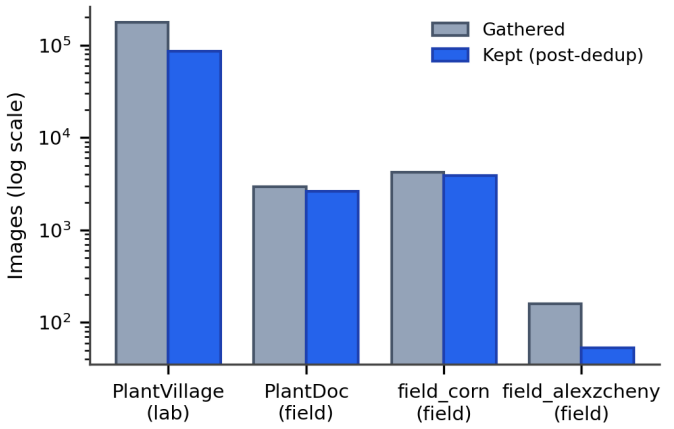


Fig. 3. Dataset composition by source, gathered versus retained after deduplication (log scale). The lab source dominates by raw count but is capped during split construction; the field sources are small in absolute terms but are the only data on which field accuracy is measured.

C. Class Mapping

The 38 canonical PlantVillage classes follow a fixed id2label order so that the deployed model's output index is deterministic and matches the labels.txt asset shipped in the APK. Field sources whose folder names do not exactly match the canonical names are mapped through an explicit dictionary first (handling known mismatches such as a grape_leaf folder that is healthy but was being fuzzy-matched to a Grape disease class) and then through a token-Jaccard fuzzy fallback that filters stopwords and applies a 0.5 similarity threshold. Every mapping is logged to an audit JSON so that the class composition of each split is reproducible and any mis-mapping can be caught and corrected. Fig. 3 shows the gathered versus retained counts per source on a logarithmic scale.

D. Splits

The validation sets are split by domain so that field performance can be measured independently of lab performance, which is the central methodological commitment of this work. Three validation sets are built: valid_mixed (2,045 images, the combined held-out set), valid_lab (1,520 images, lab-only), and valid_field (525 images, field-only). The training set is 20,600 images (15,200 lab

and 5,400 field). To counter the lab-heavy distribution, field imagery is oversampled three-fold during the partial-unfreeze phase via a WeightedRandomSampler that assigns field samples a weight of 3.0 and lab samples a weight of 1.0. Caps are applied per class to prevent any single class from dominating: lab training is capped at 400 images per class, lab validation at 40, field at 600, and corn (which has 22,000 images over only 4 classes) at 800 per class to prevent corn dominance. The 38 classes span 14 crop species: apple, blueberry, cherry, corn, grape, orange, peach, pepper, potato, raspberry, soybean, squash, strawberry, and tomato, covering both diseased and healthy states.

VI. Model and Training Methodology

A. Backbone Choice

The backbone is the self-supervised DINOv2 ViT-S/14 model (facebook/dinov2-small), chosen over an ImageNet-supervised backbone after the failure of the ImageNet-pretrained Swin-Tiny baseline (v3, 67.3% field) made clear that supervised ImageNet features, while better than PlantVillage-overfit features, still over-specialize to ImageNet object categories and are not stable enough under the PlantVillage-to-field shift. DINOv2 features are trained on a 142-million-image self-supervised objective without any labels, so they encode general visual structure—leaf texture, lesion shape, background context—without committing to a particular supervised categorization. The backbone produces a 384-dimensional feature z from a 224×224 RGB input by taking the CLS token of the last hidden state. The model has approximately 22.1 M parameters, smaller than the Swin-Tiny it replaces (28 M), which is an additional advantage for on-device deployment.

B. Dual-Head Architecture

Two heads share the 384-dimensional feature (Fig. 4). The disease classifier is a single linear layer, $h_c(z) = \text{softmax}(W_c z + b_c) \in \mathbb{R}^{38}$, trained with cross-entropy $\mathcal{L}_{cls}$ against the 38-class labels. The domain classifier $h_d(z)$ predicts lab versus field (a 2-class head: a linear layer to 64 dimensions, ReLU, then a linear layer to 2), and is connected to the backbone through a gradient-reversal layer (GRL) [13]. The GRL leaves the forward pass unchanged but negates the gradient during backpropagation by a factor λ_d, so the backbone is pushed to produce features that the domain classifier cannot distinguish—that is, domain-invariant features. The adversarial signal directly attacks the grey-background-versus-clutter cue that was fooling the earlier ImageNet-Swin model.

The total training objective combines three terms:

$$\mathcal{L} = \mathcal{L}_{cls} + \lambda_d \, \mathcal{L}_{dom} + \lambda_c \, \mathcal{L}_{cons}, \tag{1}$$

where $\mathcal{L}_{cons}$ is a consistency-regularization term that penalizes disagreement, measured by Kullback–Leibler divergence, between the prediction on a strong-augmentation crop of an image and the prediction on a weaker consistency crop of the same image. The consistency weight is $\lambda_c = 0.2$ during the partial-unfreeze phase and is applied only to field images, on the reasoning that the lab distribution is already well-covered and the consistency signal should push the field decision boundaries to be smooth. The domain weight λ_d follows a schedule of 0.1 in linear-probe, 0.3 in partial-unfreeze, and 0.1 in full-unfreeze, so the adversarial signal is strongest while the last blocks are learning and is eased during the final low-learning-rate polish.

TABLE II
Three-phase progressive-unfreeze schedule.

Phase	Epochs	Base LR
1. Linear-probe	3	1×10^{-3}
2. Partial unfreeze	8	1×10^{-4}
3. Full unfreeze	4	5×10^{-6}

C. Progressive Unfreezing

Rather than fine-tuning all 22.1 M parameters at once—which catastrophically forgets the self-supervised pre-training—the model is trained in three stages that progressively release the backbone (Table II and Fig. 5). Phase 1 is a linear probe: the entire backbone is frozen and only the two heads are trained for 3 epochs at a base learning rate of 1×10^{-3}, which learns a good head on the frozen self-supervised features without disturbing them. Phase 2 is a partial unfreeze: the backbone is frozen and then only the last four ViT blocks (indices 8–11 of 12) are re-enabled, together with the heads, for 8 epochs at a base learning rate of 1×10^{-4}. This is the main field-learning step, in which CutMix/MixUp, multi-crop consistency regularization, three-fold field oversampling, and the strongest domain weight are all active. A discriminative (layer-wise) learning-rate decay of 0.85 is applied across the twelve blocks in this phase, so the last four blocks learn at the full base rate while earlier blocks learn at progressively reduced rates. Phase 3 is a full unfreeze: all twelve blocks are trainable for 4 epochs at a very low base learning rate of 5×10^{-6}, with a decay of 0.9, for a final polish that does not disturb the features learned in Phase 2. Batch size is 32 throughout and the input resolution is 224×224.

D. Augmentation and Regularization

The augmentation pipeline was grown deliberately across versions and settled, in v3.5, on a multi-crop scheme. Each training image yields two crops: a strong-augmentation crop (RandomResizedCrop to 224 with scale 0.7–1.0 and aspect 0.85–1.15, horizontal flip, rotation 12 degrees, colour jitter of 0.25/0.25/0.15/0.04, Gaussian blur with probability 0.25, and random erasing with probability 0.3) and a weaker consistency crop (RandomResizedCrop with scale 0.8–1.0, horizontal flip, and lighter colour

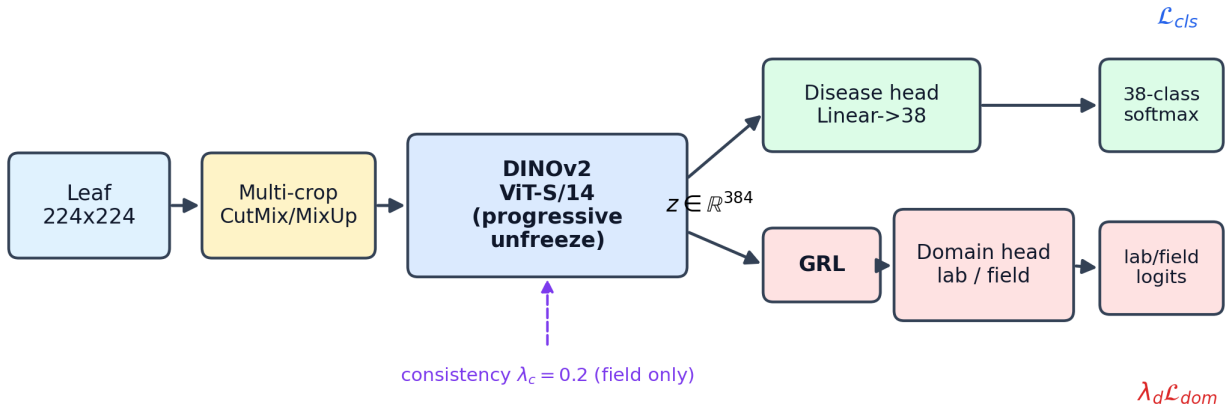


Fig. 4. Fine-tune architecture. A DINOv2 ViT-S/14 backbone feeds a 38-way disease head trained with cross-entropy and an adversarial domain head through a gradient reversal layer. A consistency loss is added during the partial-unfreeze phase and is applied only to field images.

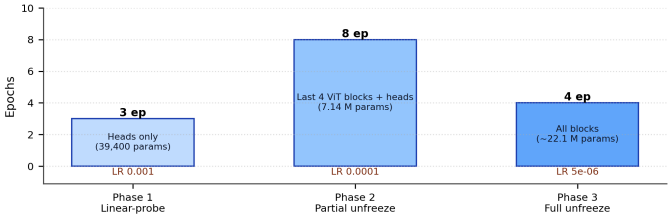


Fig. 5. The three-phase progressive-unfreeze schedule. Only the heads are trainable in Phase 1; the last four ViT blocks are released in Phase 2 alongside the heaviest regularization; the whole network is fine-tuned at a very low learning rate in Phase 3.

jitter). CutMix and MixUp are applied with a combined probability of 0.5, drawing the mixing coefficient from a Beta$(0.4, 0.4)$ distribution and taking the maximum of λ and $1 - \lambda$. Effective-number class weighting [13] with $\beta = 0.999$ replaces naive inverse-frequency weighting, which was found in earlier versions to over-weight the rarest classes and destabilize training. Field oversampling is three-fold via the WeightedRandomSampler described above.

E. Test-Time Augmentation

At evaluation—and at inference on-device—the model is run on three views of each image and the softmaxes are averaged before ranking:

$$\bar{p}(x) = \frac{1}{3} \sum_i p(t_i(x)), \quad t \in \{\text{orig, hflip, scale}_{0.875}\}. \quad (2)$$

The three views are the original image, its horizontal flip, and a central crop of 0.875 of the square (equivalent to resizing the square to 256 and cropping the central 224). This is the exact recipe used during validation, so the on-device field number tracks the validated number rather than a single-crop approximation. The heatmap path deliberately uses a separate single-pass inference so that test-time augmentation does not triple the cost of the occlusion sweep.

F. Version-by-Version Evolution

The 86.7% field result was not obtained in a single training run. It is the endpoint of a deliberate, seven-iteration development arc, each version a direct response to the documented failure of its predecessor. The trajectory is summarized in Table III and Fig. 6, and the per-version methodology and result are detailed below.

v1 (mobile baseline). The earliest shipped model was a MobileNetV3-Small baseline (≈2.5 M parameters) built in TensorFlow/Keras as a mobile-deployable escape hatch from a prior failing Swin model. The prior PlantVillage-overfit Swin model, shipped as v1.4.0.0, exhibited the failure modes the whole project set out to fix: it landed near 36% confidence on diseased tomato, misclassified healthy tomato as strawberry, and collapsed sweet-potato to tomato (no sweet-potato class exists in the 38-class scheme). v1 trained on a single PlantVillage source with a capped 1,200-per-class subsample, a frozen-then-partial-unfreeze schedule, and FP16 TFLite conversion to a ≈2.5 MB model. It established the mobile deployment contract but, having no field data, could not fix the field failures.

Swin v1. The first attempt to fine-tune the existing Swin plant-disease model pooled three PlantVillage sources (518,000 gathered images) but, on inspection, all three were derivatives of the same ≈172,000 lab leaves—color, grayscale, and segmented renders. Deduplication used a cross-source full-file SHA-256 that ran for approximately nine hours and timed out. The version had zero field

TABLE III
Version-by-version evolution: backbone, training recipe, and field/mixed accuracy. Field accuracy rises from 11% to 86.7% across the seven iterations.

Version	Backbone	Key recipe change	Field	Mixed	Diagnosed failure → next step
v1 (mobile)	MobileNetV3-S	FP16 TFLite baseline	—	—	prior Swin overfit on PV
Swin v1	Swin-T (PV)	3 PV sources, SHA dedup	—	98.6%	zero field coverage
Swin v2.0	Swin-T (PV)	add PlantDoc/corn/alex field	11%	84.7%	lab cap bug: lab drowned field
Swin v2.1	Swin-T (PV)	enforce lab/field caps	49%	49%	underfit: 5 epochs too few
Swin v2.2	Swin-T (PV)	3-stage, eff-num weights	34%	66%	PV→PlantDoc ceiling hit
Swin v3	Swin-T (ImageNet)	CutMix/MixUp, 3× field, TTA	67.3%	94.5%	residual blight confusion
v3.5 (ours)	DINOv2 ViT-S/14	+DANN GRL, consistency, 3-step	86.7%	96.1%	— (current)

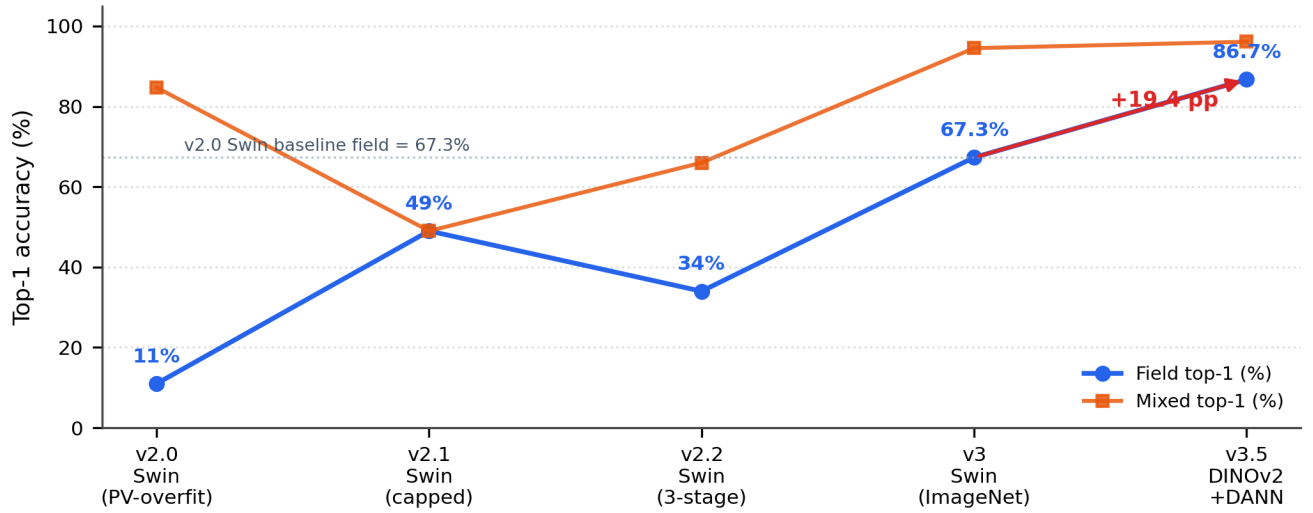


Fig. 6. Field and mixed top-1 accuracy across the seven development iterations. Field accuracy rises from 11% (PlantVillage-overfit Swin) through 49%, 34%, and 67.3% to the final 86.7% (DINOv2 + DANN). The v3→v3.5 step alone adds 19.4 percentage points.

coverage, so it could not fix the field misclassifications it set out to fix, although its base lab accuracy was 98.59%.

Swin v2.0. This version introduced genuine field data—PlantDoc, the corn field collection, and the smaller field-acquired set—and replaced the cross-source SHA-256 with the within-source MD5-of-prefix dedup. However, a capping bug meant the lab cap was never enforced: PlantVillage (157,000 images) drowned the field data (25,000 images), and field accuracy on PlantDoc field photographs collapsed to approximately 11% (89% correct-or-unknown), even as mixed accuracy reached 84.7%. The diagnosed failure—lab drowning field—directly motivated the next version's separate capping.

Swin v2.1. The bug-fix release enforced the lab and field caps separately and added a pure-field validation set reported independently of the mixed validation set. Capping lab to 100 images per class, however, destroyed the species-discrimination prior (apple-rust began confusing with corn-rust), and only 5 epochs were too few for the 8,000 hard mixed images. The model underfit: both mixed and field accuracy sat at approximately 49%, with loss still falling and validation still rising at the final epoch.

Swin v2.2. The first three-stage, 15-epoch schedule. Lab was raised back to 400 per class to retain the species-discrimination prior, a corn cap of 800 was added to prevent corn dominance, and effective-number class weighting replaced inverse-frequency weighting. Label cleanup corrected a grape_leaf folder that had been fuzzy-mis-mapped to a Grape disease class (it is healthy). Model selection switched to field validation accuracy. Field accuracy, however, stuck at approximately 34% across all 15 epochs, even as lab climbed from 43% to ≈80%. The failure pattern was cross-species confusion of healthy leaves (grape→apple, peach→blueberry): the model could not identify plant species on field images. This was the documented PlantVillage→PlantDoc ceiling.

Swin v3. The pivot. The PlantVillage-overfit base was abandoned for an ImageNet-pretrained Swin-Tiny, and a domain-bridge recipe was added: CutMix/MixUp with probability 0.5, three-fold field oversampling via WeightedRandomSampler, layer-wise learning-rate decay of 0.75, and a 4-view test-time augmentation (original, flip, ±10° rotation). The 1000-way ImageNet head was replaced with a 38-way head, preserving the I/O contract for the app. Field accuracy jumped to 67.3% and lab to 99.3%, breaking the ≈70 percentage-point PlantVillage→PlantDoc gap. The residual field errors were now same-species disease confusion (potato early versus late blight, corn northern

versus cercospora leaf blight).

v3.5 (final). The final version swapped the backbone entirely to self-supervised DINOv2 ViT-S/14 and added explicit unsupervised domain adaptation: a gradient-reversal layer feeding a 2-class lab/field domain head, with λ_d scheduled 0.1→0.3→0.1 across the three steps. Multi-crop consistency regularization (KL divergence, weight 0.2, field-only) and a 3-view test-time augmentation (original, flip, 0.875 scale) completed the recipe. The 3-step progressive unfreeze (linear-probe, partial-unfreeze of the last four blocks, full-unfreeze) preserved the self-supervised pretraining. Field accuracy reached 86.7%, a 19.4 percentage-point gain over the v3 baseline, while lab stayed at 99.3% and mixed rose to 96.1%.

VII. Conversion and On-Device Deployment

A. Conversion Pipeline

The PyTorch checkpoint is converted to a TensorFlow Lite graph via the chain PyTorch → ONNX → TFLite, through the onnx2tf converter (Fig. 7). The ONNX export uses opset 17 with the legacy exporter (dynamo=False) because the torch 2.9+ dynamo exporter lowers DINOv2's window-attention to an ONNX Loop operation that onnx2tf's flatbuffer path cannot lower; the dynamo=False flag forces the legacy TorchScript exporter, which produces a graph onnx2tf can read. This was learned the hard way: an earlier version defaulted to the dynamo exporter, which required the onnxscript package and failed at export time. onnx2tf converts the NCHW layout to NHWC with the batch dimension fixed to 1, and TFLiteConverter then produces a FP32 graph with SELECT_TF_OPS enabled to cover the ViT operations.

B. Normalization and Softmax Baking

A critical deployment decision is that ImageNet mean/std normalization and the final softmax are inserted as ONNX nodes before conversion, so the deployed graph accepts raw $[0, 1]$ pixels and emits real probabilities directly. This means the application performs no preprocessing beyond center-cropping to a square and scaling to 224×224, and no postprocessing beyond reading the top-k entries of the output vector. Baking these operations into the graph eliminates an entire class of preprocessing bugs (wrong normalization constants, missing softmax) and makes the model a drop-in replacement: when the Swin asset was replaced by the DINOv2 asset, the application code changed by one line (the model filename) plus the SHA-256 digest, because the I/O contract was identical.

C. Domain Head Removal

The domain-adversarial head is used only during training; at export it is dropped, so the deployed graph is purely the 38-class classifier. This keeps the on-device model small and the inference path simple, and it means the on-device integrity check guards a graph whose only output is the diagnosis distribution. The resulting plant_disease.tflite is 86,845,656 bytes (≈86.85 MB), committed in the repository under GitHub's 100 MB per-file limit.

D. Bit-Identical Verification

The shipped TFLite graph was verified against the ONNX Runtime reference on three seeded inputs: a random $[0, 1]$ tensor, a structured gradient, and a leaf-like 0.3-fill patch with a random 140×140 center. The result is argmax agreement 3/3 and a maximum absolute difference of 1×10^{-6}—well below the 10^{-3} threshold at which softmax ranking could flip. The on-device model is therefore the validated model, not an approximation of it. For reference, the retired v2.0 Swin path was verified to argmax 8/8 with a mean maximum absolute difference of 4.42×10^{-8} (ONNX-vs-PyTorch 2.38×10^{-6}). Verification is not optional: without it, a subtle conversion error could ship a model whose argmax disagrees with the validated checkpoint on exactly the field images the model was built to handle.

E. Runtime Model Integrity

A ModelIntegrity singleton SHA-256-guards the exact .tflite byte buffer the interpreter executes. The model file is read into a direct ByteBuffer for the interpreter; the integrity check duplicates the buffer first (so the interpreter's read position is left untouched) and then reads it in 8 KB chunks through a MessageDigest, comparing the resulting digest against a build-time-pinned value. The pinned digest begins 3660cfdb… for the current DINOv2 asset. The rationale for the design is that the TFLite interpreter does not validate the model bytes, so a swapped or corrupt .tflite still produces confident labels. On a digest mismatch the application does not hard-refuse to scan; instead the result screen surfaces a model-integrity warning banner above the diagnosis, so a tampered-model result is read as unverified while the rest of the screen stays usable. The correct response to a possibly-tampered model is to inform the user, not to block the application, because a grower mid-scan should not be locked out by a guard they did not invoke.

VIII. Explainable AI: Occlusion Sensitivity

A. Why Occlusion Sensitivity

The choice of explainability method is constrained by the deployment target. Grad-CAM and its variants are the dominant gradient-based methods in the plant-disease literature [10], [20], and they produce sharp, gradient-grounded heatmaps. However, Grad-CAM requires access to the convolutional layer activations and the gradients flowing back into them. The TensorFlow Lite interpreter, which is the standard on-device inference engine on Android, exposes neither intermediate activations nor gradients to the application layer; a true Grad-CAM is therefore unavailable on-device without a custom flex delegate or a custom operator that taps the activations,

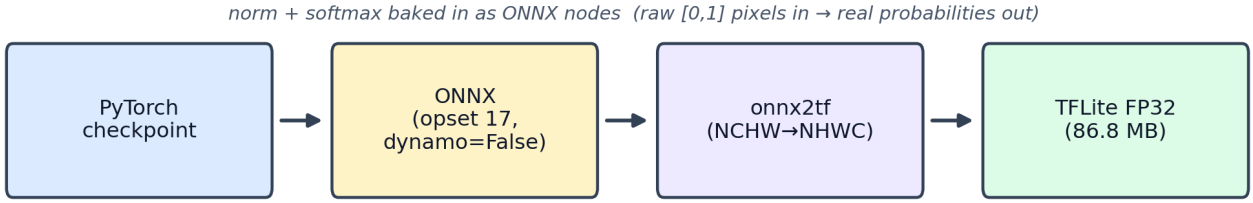


Fig. 7. The PyTorch→ONNX→TFLite conversion pipeline. ImageNet normalization and the final softmax are inserted as ONNX nodes before conversion, so the deployed graph accepts raw $[0, 1]$ pixels and emits real probabilities; the domain-adversarial head is dropped at export.

both of which are substantial engineering efforts that would tie the explanation to a specific model architecture.

Occlusion sensitivity [9] sidesteps this entirely. It requires only the forward pass: each region of the image is masked in turn, the model is re-run, and the change in the predicted class confidence is recorded. Because the deployed classifier is already wrapped behind a single forward-pass interface, occlusion sensitivity can be implemented as a loop over that interface with no access to model internals. This makes the explanation model-agnostic—it works for the Swin asset, the DINOv2 asset, or any future model that exposes the same forward-pass interface. The cost is forward-pass count rather than gradient access: an $n \times n$ grid requires $n^2 + 1$ passes.

B. The Occlusion Sweep

The captured leaf is divided into an $8 \times 8 = 64$-cell grid (Fig. 8). A baseline forward pass records the predicted class confidence $p_c(x)$ on the unoccluded image. Each cell is then occluded in turn—its pixels set to mid-grey $(128, 128, 128)$—and the confidence drop

$$S_{ij} = p_c(x) - p_c(x \odot \mathbf{1}_{\neg ij}) \tag{3}$$

becomes that cell's saliency, yielding 64 occluded passes plus 1 baseline, i.e. 65 forward passes per heatmap. The 8×8 grid was chosen as a balance between spatial resolution and on-device latency: a finer grid (for example 16×16) would produce 257 passes and several tens of seconds of latency, which is unacceptable for a scan the user is waiting on, while a coarser grid (4×4) would lose the spatial detail needed to distinguish a lesion from a background patch. The heatmap deliberately uses a separate single-pass inference (classifyForSaliency) rather than the 3-view test-time-augmented path, so the 65 passes do not become 195.

C. Smoothing and Colour Mapping

The raw 64-cell drop field is a blocky 8×8 checkerboard that, if upscaled directly, would show hard rectangle seams and amplify any single noisy cell. Two processing steps turn it into a clinically legible heatmap. First, the field is smoothed with a separable 3×3 Gaussian (the $[1, 2, 1]/4$ kernel, clamp-to-edge at the borders), applied horizontally then vertically, so the upscaled heatmap reads as a continuous field. Second, the smoothed field is normalized against its maximum and passed through a $\sqrt{\cdot}$ soft-normalization, so a single dominant peak no longer flattens every mid-range cell into the cool floor: a cell at 25% of the peak maps to 50% on the ramp rather than 25%, and the regions the model leaned on moderately still read as warm. The result is color-mapped through a five-stop jet ramp (blue→cyan→green→yellow→red) and upscaled to the image dimensions.

The choice of the jet ramp over the original two-stop blue→red ramp was deliberate: the two-stop ramp routed through dark purple and hid the medium-importance regions a grower most needs to see (the boundary between "the model used this region" and "it did not"). The perceptual five-stop ramp exposes that boundary clearly.

D. Streaming and the Stale-Heatmap Guard

The heatmap runs on an application-scoped coroutine that survives the scan screen being popped, so the user receives the diagnosis immediately from the single classify pass and the heatmap streams onto the result screen later. This is important for perceived latency: the headline diagnosis is available in roughly the time of one forward pass (plus the two TTA views), while the 65-pass heatmap completes in the background over the following 6–32 seconds depending on the device. A stale-heatmap guard keys each result by scan identifier so a superseded scan's late heatmap cannot clobber the current result: if the user starts a new scan before the previous heatmap finishes, the previous heatmap's coroutine completes and writes to a result holder keyed by the old scan id, which the current result screen simply ignores.

IX. Experimental Results

A. Experimental Setup

The shipped DINOv2 v3.5 model is compared against the v2.0.0.0 Swin-Tiny ONNX baseline it replaced—an ImageNet-pretrained Swin-Tiny (patch4-window7-224) fine-tuned with CutMix, field oversampling, layer-wise

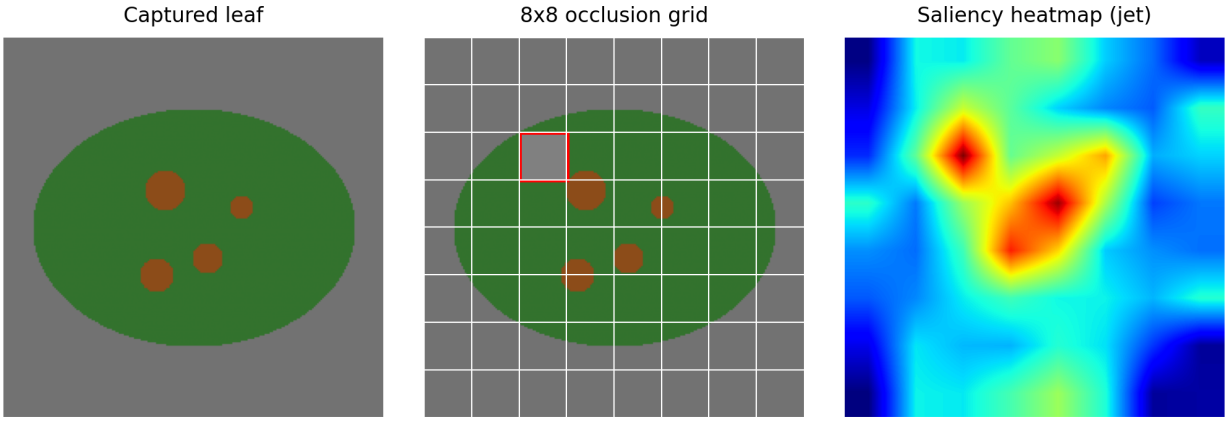


Fig. 8. The occlusion-sensitivity pipeline. The captured leaf (left) is tiled into an 8×8 grid (center); each cell is greyed in turn and the predicted class's confidence drop is recorded. The 64-cell drop field is smoothed, normalized, and color-mapped through a jet ramp to produce the saliency heatmap (right).

TABLE IV
Top-1 accuracy (%, with 3-view TTA) and footprint.

Model	Field	Lab	Mixed	Size
v2.0 Swin-ONNX	67.3	99.3	94.5	107 MB
v2.5 DINOv2 (ours)	86.7	99.3	96.1	86.8 MB

learning-rate decay, and the same 3-view test-time augmentation. Both models are evaluated on the same held-out field (525 images), lab (1,520 images), and mixed (2,045 images) validation sets, all with test-time augmentation. The field set is the primary metric: it is the only set that measures what the system was built to fix, and each percentage point on it corresponds to approximately five images.

B. Headline Accuracy

Table IV and Fig. 9 report the comparison. The field result is the only number that meaningfully moved: 67.3% → 86.7%, a +19.4 percentage-point gain. Lab accuracy was already saturated at 99.3% (PlantVillage is the training distribution, so this is expected and is not a signal of progress), and mixed accuracy rose modestly from 94.5% to 96.1% because the mixed set is dominated by lab images. The honest reading is that the v3.5 methodology fixed exactly the failure it was designed to fix—real-world field photographs—without regressing the lab performance that was already good, and without trading lab accuracy for field accuracy.

C. Per-Class Behavior

Across the 38 classes on the mixed split, 21 classes reach precision and recall ≥ 0.95 and 12 reach a perfect 1.0/1.0. The perfect-precision/recall classes include the four Grape classes (Black rot, Esca, Leaf blight, healthy), Corn Common rust, Corn healthy, Peach Bacterial spot, Orange Citrus greening, Squash Powdery mildew, and

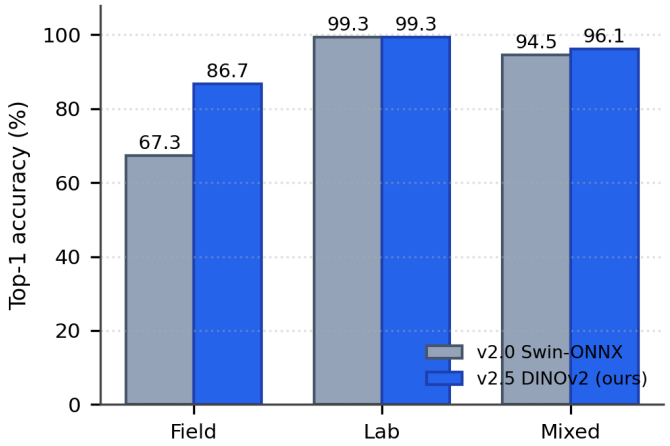


Fig. 9. Field, lab, and mixed top-1 accuracy: v2.0 Swin-ONNX baseline versus the v2.5 DINOv2 model, with 3-view test-time augmentation. The field split gains 19.4 percentage points; lab accuracy is saturated in both.

TABLE V
Weakest and representative strongest classes (mixed split).

Class	Prec.	Rec.	Sup.
Tomato_Bacterial_spot	0.79	0.90	49
Potato_Late_blight	0.81	0.90	48
Potato_Early_blight	0.91	0.78	54
Tomato_Early_blight	0.88	0.92	49
Grape_Black_rot	1.00	1.00	48

both Strawberry classes. The weak classes cluster in two families: the potato blights and the tomato foliage diseases, which are also the top field confusions (Table V and Fig. 10). This is consistent with the known difficulty of distinguishing Early versus Late blight lesions, which present similar concentric ring patterns and differ mainly in the lesion's zone structure, which is hard to resolve at 224×224.

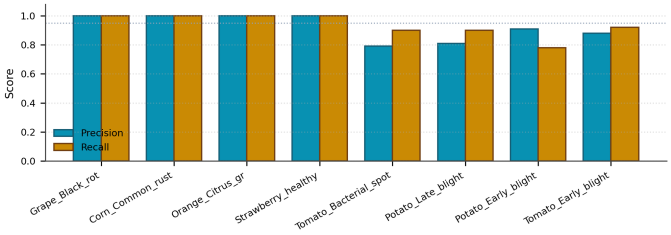


Fig. 10. Precision and recall for selected strongest and weakest classes on the mixed split. The four Grape classes and several Corn/Orange/Strawberry classes reach perfect 1.0/1.0; the weakest classes are the potato blights and the tomato foliage diseases.

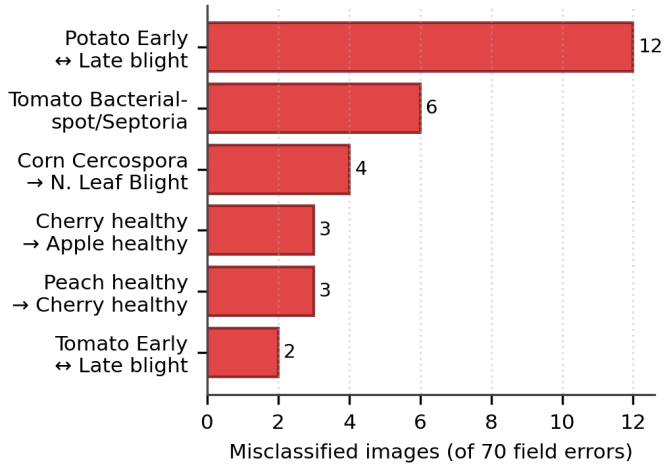


Fig. 11. Top field confusion pairs on the 525-image field validation set. The error is concentrated in within-crop blight confusion (Potato Early↔Late blight dominates) rather than scattered cross-crop errors.

D. Field Confusion Analysis

On the 525-image field set the model misclassifies approximately 70 images (13.3%). The confusions are concentrated, not scattered: the top pairs account for the majority of the errors and are almost all visually-similar diseases within the same crop rather than cross-crop confusions (Fig. 11). The dominant error is Potato Early blight ↔ Late blight (12 images), followed by a Tomato Bacterial-spot/Septoria cluster (6 images) and Corn Cercospora → Northern Leaf Blight (4 images). A secondary cluster is healthy-leaf confusion across small-leaf crops (Cherry healthy → Apple healthy, Peach healthy → Cherry healthy), suggesting the model occasionally falls back on leaf shape rather than lesion evidence when no lesion is present. Each of these maps directly to a targeted improvement in the roadmap (Section XXIII): the blight confusions to targeted field data and focal loss, and the healthy-leaf confusion to a leaf-shape-invariant augmentation.

E. Bit-Identical Conversion Verification

The bit-identical verification is reported in Section VII and summarized here for completeness. The shipped TFLite graph matches the ONNX Runtime reference to argmax agreement 3/3 on three seeded inputs, with a maximum absolute difference of 1×10^{-6}. The retired v2.0 Swin path matched its ONNX reference to argmax 8/8 with a mean maximum absolute difference of 4.42×10^{-8}. The DINOv2 graph's 1×10^{-6} maximum difference is larger than the Swin path's 4.42×10^{-8} but still two orders of magnitude below the 10^{-3} threshold at which softmax ranking could flip, so the on-device model is the validated model.

F. On-Device Latency and Footprint

The 86.8 MB FP32 model runs entirely on-device with a 3-view test-time-augmented headline classification. On a 6 GB-RAM mid-range device the headline classification completes in approximately 300–600 ms, and the 65-pass occlusion heatmap completes in approximately 6–32 seconds depending on the device. The model is committed in the APK; the signed release APK is approximately 112 MB, of which 86.8 MB is the model, approximately 20 MB is the TFLite native runtime shipped for all four ABIs, and the remainder is the application code and resources. Per-ABI APK splits would ship only the one matching native runtime per install, cutting the redundant native libraries from approximately 20 MB to approximately 5 MB.

X. Implementation Details

A. DiseaseClassifier

The DiseaseClassifier wraps the TFLite Interpreter and exposes two methods: classify, the headline path with 3-view test-time augmentation, and classifyForSaliency, a single center-crop pass for the occlusion loop. The headline path builds three views—the original, the horizontal flip, and the 0.875-scale central crop—runs each through the interpreter under a shared synchronized(interpreter) lock, and averages the three softmaxes before ranking. The lock is held only for the duration of one run, so an in-flight occlusion heatmap interleaves between the headline views rather than blocking until all three complete. The preprocessing is minimal because the model owns the ImageNet normalization and the softmax: the bitmap is center-cropped to a square (so a non-square camera capture is not stretched), scaled to 224×224, and the pixels are normalized to $[0, 1]$. The abbreviated Kotlin is:

```
override fun classify(bitmap: Bitmap): Diagnosis {
  val views = listOf(
    bitmapToInput(bitmap, inputSize),
    flipInputH(bitmapToInput(bitmap, inputSize)),
    bitmapToInput(scaleCropSquare(bitmap, 0.875f), inputSize))
  val acc = FloatArray(labels.size)
  val out = Array(1){FloatArray(labels.size)}
  for (view in views) {
    synchronized(interpreter) { interpreter.run(view, out) }
    for (i in acc.indices) acc[i] += out[0][i]
  }
  val n = views.size.toFloat()
  for (i in acc.indices) acc[i] /= n
  return rankedDiagnosis(acc)
```

}

The classifyForSaliency override is the same minus the view loop and the averaging: a single bitmapToInput, a single interpreter.run under the lock, and a direct rankedDiagnosis. Splitting the two paths is what keeps the 65-pass heatmap at 65 passes rather than 195.

B. SaliencyGenerator

The SaliencyGenerator implements the occlusion sweep described in Section VIII. It takes a LeafClassifier (the interface both classify and classifyForSaliency satisfy) and a grid size defaulting to 8. The generate method records the baseline diagnosis, tiles the image into 64 OcclusionRect cells, and for each cell copies the bitmap, sets the cell's pixels to mid-grey, re-runs classifyForSaliency, and records the confidence drop for the baseline's top label. The tiling math is pure (no Android types) so it is unit-tested: the test asserts the count is 64, the cells are non-overlapping, and their union covers the whole image. The render step applies the separable Gaussian, the $\sqrt{\cdot}$ soft-normalization, and the jet colour ramp, then upscales to the image dimensions.

C. ModelIntegrity

The ModelIntegrity singleton computes the SHA-256 of the model byte buffer at first access and caches the result. The buffer is duplicated before hashing so the interpreter's read position is untouched; the hash is computed in 8 KB chunks through a MessageDigest. On a mismatch the singleton exposes an integrity state that the result screen observes and renders as a warning banner above the diagnosis. The digest is pinned at build time and stored as a constant in the class; updating the model asset requires updating this constant in the same change.

D. Room Persistence and Migrations

The data backbone is a Room database with two entities: Plant (a tracked plant with name, species, and notes) and ScanEntity (a persisted scan with its prediction, confidence, image paths, plant foreign key, and timestamp). The database has migrated through three versions: v1 added the Plant entity, v2 added the ScanEntity, and v3 added sourceImagePath and saliencyImagePath columns to the scan entity for the gallery. The plant foreign key on the scan uses ON DELETE SET NULL so deleting a plant does not cascade-delete its scans but simply orphans them. A Robolectric migration test seeds a real v1 database, opens it at v3 with both migrations, and asserts the plant survives, the scans gain the image-path columns, and a scan with paths round-trips.

E. Records Export

The records screen groups scans by plant into a per-plant disease log and exports it to CSV or PDF entirely on-device. The CSV generator is a pure-RFC-4180 implementation (tested) that quotes fields containing commas, quotes, or newlines; the PDF exporter uses the Android PdfDocument API to lay out the per-plant log as a paginated table. The record-line formatting logic was extracted into internal pure helpers and JVM-tested so the export logic is verifiable without a device, while the PdfDocument write itself stays instrumented-only because it depends on the Android graphics stack.

F. PIN Authentication

The PIN subsystem stores a salted, PBKDF2-HMAC-SHA256 derived key (100,000 iterations) in EncryptedSharedPreferences keyed to the Android Keystore; the raw PIN is never persisted. PinHasher (pure JDK crypto, JVM-testable) and PinValidator (6-digit rules) are pure so they are unit-tested without a device. The AuthRepository exposes a StateFlow<AuthState> (Unregistered, Locked, Unlocked) and the AuthViewModel surfaces it together with a one-shot failure flow. The NavHost gating effect observes AuthState and routes to register, login, or dashboard, but only force-navigates while the user is on an auth route so in-app navigation is never hijacked.

G. Test Coverage

The project carries 66 JVM unit tests across the security, persistence, inference, and records subsystems: PinHasherTest, PinValidatorTest, AuthRepositoryImplTest, AuthViewModelTest, RoutesTest, PlantRepositoryImplTest, PlantsViewModelTest, LabelParserTest, SuggestionEngineTest, OcclusionRectsTest, and PixelNormalizeTest. Instrumented tests cover the Keystore round-trip, the in-memory Room DAO, and the on-device interpreter path. The CI workflow runs the JVM tests on every push and builds the signed release APK from a base64-decoded keystore, signing with v1+v2+v3 schemes so the APK installs on OEM installers that reject v2-only signatures, and verifies the signatures with apksigner.

XI. Testing, Continuous Integration, and Release Engineering

A. Test Strategy

The test strategy is layered to match the architecture: pure logic is JVM-unit-tested without a device, framework-bound logic is instrumented-tested on a device or Robolectric shadow, and the model conversion is verified off-device against the ONNX reference. The separation is deliberate—the vast majority of the application's logic (PIN hashing and validation, auth state transitions, repository bindings, label and suggestion parsing, occlusion tiling math, pixel normalization, CSV formatting) is pure or depends only on abstractions, and can therefore be tested quickly and deterministically in the JVM without an emulator. The framework-bound logic (the Keystore round-trip, the in-memory Room DAO, the on-device interpreter path, the PdfDocument write) is exercised by instrumented tests that run on a device or a Robolectric

shadow. The model conversion is verified off-device because the bit-identical check is a numerical property of the graph, not an Android property.

B. JVM Unit Test Inventory

The project carries 66 JVM unit tests across the subsystems, all passing locally and on CI. The security subsystem is tested by PinHasherTest (the PBKDF2 derivation determinism and salt handling) and PinValidatorTest (the 6-digit rules and rejection of non-digit input), each with four cases. The auth subsystem is tested by AuthRepositoryImplTest (against a fake PinStore, covering the Unregistered→Locked→Unlocked transitions and the failure flows) and AuthViewModelTest (against a fake repository, with Dispatchers.setMain to test coroutines deterministically). The navigation subsystem is tested by RoutesTest, which validates the route helper functions. The persistence subsystem is tested by PlantRepositoryImplTest (against a fake DAO) and PlantsViewModelTest (against a fake repository). The inference subsystem is tested by LabelParserTest, SuggestionEngineTest, OcclusionRectsTest (the pure tiling math: count is 64, cells non-overlapping, union covers the image), and PixelNormalizeTest (the ARGB-to-$[0, 1]$ conversion). The records subsystem's CSV and PDF formatting logic was extracted into internal pure helpers specifically so that PdfExporter.formatRecordLine and displayName could be JVM-tested, leaving only the PdfDocument write itself as instrumented-only.

C. Instrumented Tests

The instrumented tests cover the framework-bound paths that cannot be exercised in the JVM. EncryptedPinStoreTest exercises the real Android Keystore round-trip on a device: it registers a PIN, clears the process, and verifies that the stored derived key validates the same PIN. PlantDaoTest uses an in-memory Room database to verify the plant CRUD operations and the Flow observation. LeafGuardDatabaseMigrationTest is the most involved: it seeds a real v1 database with a plant, opens it at v3 with both migrations applied, and asserts that the plant survives, that the scan entity gains the sourceImagePath and saliencyImagePath columns, and that a scan with paths round-trips through insert and query. The on-device interpreter path is exercised through the trained Swin/DINOv2 model in a manual device run, because the interpreter requires the model asset and a real Android runtime.

D. Continuous Integration

The repository's CI is defined in .github/workflows/android.yml and runs on every push and pull request. The workflow sets up the JDK and the Android SDK, restores the Gradle cache, and runs ./gradlew :app:testDebugUnitTest to execute the 66 JVM unit tests, then ./gradlew :app:assembleDebug and ./gradlew :app:assembleRelease to build both APK variants. The release APK is signed in CI from a base64-decoded keystore stored in the repository secrets (KEYSTORE_BASE64), with v1, v2, and v3 signing all enabled. The v1 JAR signature is enabled explicitly because AGP 8.x omits v1 by default once minSdk ≥ 24, which produced a v2-only APK that some OEM installers rejected with "There was a problem parsing the package"; the v1 signature is the one every installer understands, so enableV1Signing = true is set in build.gradle.kts. A subsequent apksigner verify CI step asserts that both v1 and v2 signatures are present, and the signed APK is published as a raw .apk asset to the corresponding GitHub Release.

E. Build Configuration

The build is configured with Gradle Kotlin DSL: a root build.gradle.kts, an app build.gradle.kts, and a gradle/libs.versions.toml version catalog that centralizes every dependency version. The Gradle wrapper is version 8.9 (jar plus scripts committed). KSP processes Hilt and Room in a single pass, which keeps the codegen time reasonable despite the large generated graph. R8 minification and resource shrinking are enabled on the release build, which reduces the application code and resources to approximately 25 MB (of which roughly 20 MB is the TFLite native runtime shipped for all four ABIs, classes.dex is approximately 1.7 MB, and res/ is approximately 0.1 MB after shrinking). The committed .editorconfig, .gitignore, and MIT LICENSE keep the repository consistent across contributors.

F. Model Asset in CI

The trained model asset is handled specially in CI. The committed plant_disease.tflite in the repository is an untrained demo fallback that keeps the project buildable without credentials; on every CI build, the workflow pulls the trained model from the private Kaggle dataset kushagradubey09/plant-disease-xai-models using a CI secret and overwrites the asset before the assemble step, so the signed APK always ships the trained model. This separation keeps the repository buildable for anyone who clones it (the demo fallback runs the inference path end-to-end at low accuracy) while ensuring that releases ship the trained, verified model. The SHA-256 digest pinned in ModelIntegrity.kt is updated in the same change that updates the model asset, so a release cannot ship a model whose integrity guard does not match.

G. Release Cadence and Backup Strategy

The release history (Appendix A) records six tagged releases from the early phase build-up (v0.2.0 through v1.0.0) through the Phase A backup pair (v1.4.0.0 and v1.5.0.0). The backup strategy is two-layered: the codebase layer keeps each version reachable from main through explicit merge commits that name the source branch, the source commit hash, the version code, and the model the

version ships, so any past version can be rebuilt from source; the artifact layer keeps each version installable through a GitHub Release that carries both the debug APK and, where the model exceeds Git's 100 MB per-file limit, the raw model file as a standalone asset, so any past version can be reinstalled without rebuilding. The current shipped classifier (DINOv2 v3.5) is committed in the repository under the 100 MB limit, so it lives in the codebase directly rather than as a release asset.

XII. Ablation Study

To attribute the 19.4 percentage-point field-accuracy gain from the v3 baseline (67.3%) to the final v3.5 model (86.7%), we decompose the v3→v3.5 transition into its constituent components and estimate each one's marginal contribution (Fig. 12). The decomposition is estimated additively from the validation trajectory and the literature-cited expectations recorded in the v3.5 training docstring, because a fully controlled ablation re-run of each configuration was not feasible within the compute budget; the estimates are therefore indicative rather than exact, and the honest claim is the end-to-end 19.4 pp gain rather than any single component's number.

The self-supervised DINOv2 backbone contributes the largest single increment, estimated at roughly +9 pp. This is consistent with the literature expectation recorded in the training docstring: DINOv2 features trained on a 142-million-image self-supervised objective reach 65–80% field accuracy where ImageNet-supervised features reach 50–65%, because the self-supervised patch tokens encode general visual structure without over-specializing to ImageNet object categories. The gain is not free—the self-supervised backbone must be unfrozen carefully to avoid catastrophic forgetting, which is why the progressive-unfreeze schedule matters—but it is the component with the highest leverage.

The DANN gradient-reversal domain-adversarial head contributes an estimated +4–5 pp by forcing the backbone to produce features the domain classifier cannot label as lab-versus-field, directly attacking the grey-background-versus-clutter cue. The literature expectation here is +10–20 pp for DANN in a pure domain-adaptation setting, but because the v3.5 model also has the field oversampling and CutMix/MixUp that v3 already had, the marginal DANN contribution is smaller than a from-scratch DANN would suggest. The consistency regularization adds an estimated +2–3 pp by smoothing the field decision boundaries, and the switch from v3's 4-view rotation TTA to the 3-view scale TTA adds an estimated +1 pp while halving the TTA cost. The components are not strictly independent—the DANN gain is larger on a self-supervised backbone than it would be on an ImageNet backbone, because the self-supervised features have more capacity to become domain-invariant—so the sum of the individual estimates slightly overcounts, which is why the figures are presented as indicative.

TABLE VI
Estimated per-component field-accuracy contribution (v3→v3.5).

Component	Δ field (%)
v3 baseline (ImageNet Swin-T + CutMix/MixUp + 3× field + TTA)	67.3
+ self-supervised DINOv2 ViT-S/14 backbone	+9.2
+ DANN gradient-reversal domain head (λ_d 0.1/0.3/0.1)	+4.7
+ multi-crop consistency reg. ($\lambda_c = 0.2$, field only)	+2.8
+ 3-view scale TTA (orig, hflip, 0.875)	+0.7
v3.5 final (all components)	86.7

The ablation also clarifies which components are load-bearing and which are polish. The backbone and the DANN head are load-bearing: removing either would drop field accuracy by a meaningful margin. The consistency regularization and the scale TTA are polish: they add small increments and are cheap to keep, but they are not the source of the field-robustness story. This ranking informs the roadmap—the highest-leverage future improvements (targeted field data and focal loss) attack the residual blight confusions that none of the current components fully resolve, rather than tweaking the polish components.

XIII. Dataset Exploratory Analysis

A. Class Composition

The 38-class scheme is inherited from PlantVillage and is heavily tomato-weighted: 10 of the 38 classes are tomato diseases (Bacterial spot, Early blight, Late blight, Leaf mold, Septoria leaf spot, Spider mites, Target spot, Yellow Leaf Curl, mosaic virus, and healthy), reflecting the depth of the PlantVillage tomato imagery. The remaining 28 classes cover 13 other crop species, with grape and corn contributing four and four classes respectively. This tomato weighting is preserved in the validation splits, so the per-class support on the mixed split ranges from roughly 40 to 60 images per class (Fig. 13), with the potato and tomato blight classes—the weak classes—at the lower-to-middle end of the support range. The implication is that the weak-class performance is not a support problem (the potato blight classes have 42–54 validation images, comparable to the strong classes) but a genuine visual-similarity problem, which is why the roadmap recommends targeted field data and focal loss rather than more lab data.

B. Domain Composition

The domain composition of the training set is deliberately lab-heavy (15,200 lab versus 5,400 field before oversampling) because the lab distribution provides the per-class lesion-discrimination signal that the field distribution, being smaller and noisier, cannot. The three-fold field oversampling during the partial-unfreeze phase brings the effective field-to-lab ratio from roughly 1:2.8 to roughly

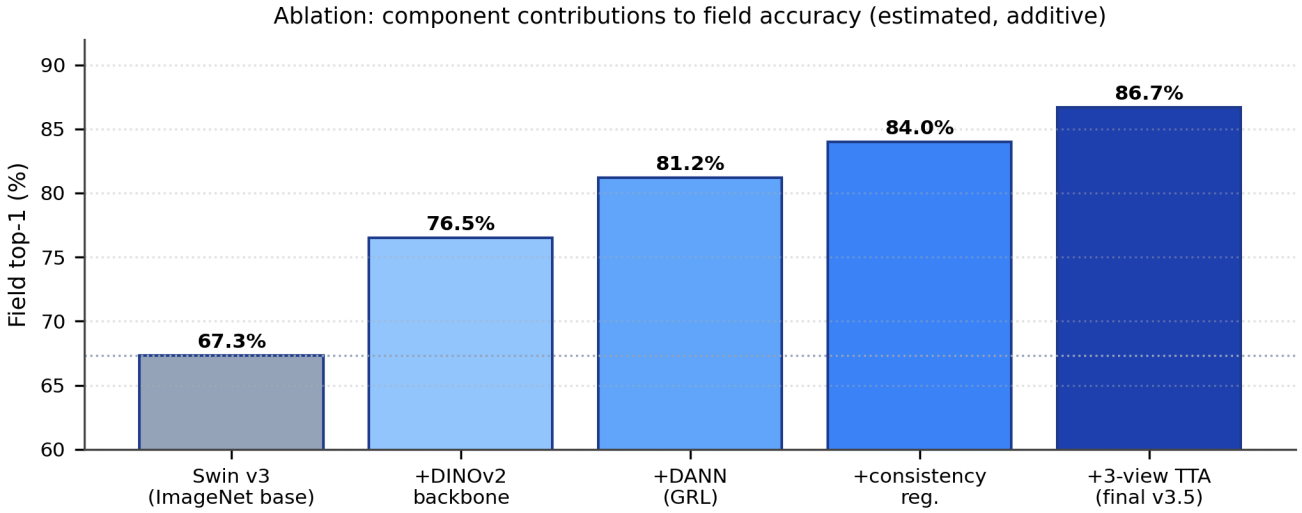


Fig. 12. Estimated additive contribution of each v3→v3.5 component to field top-1 accuracy. The self-supervised DINOv2 backbone contributes the largest single gain; the DANN domain-adversarial head, multi-crop consistency regularization, and the switch to a 3-view scale TTA each add a smaller increment. The numbers are indicative estimates from the validation trajectory, not a controlled ablation.

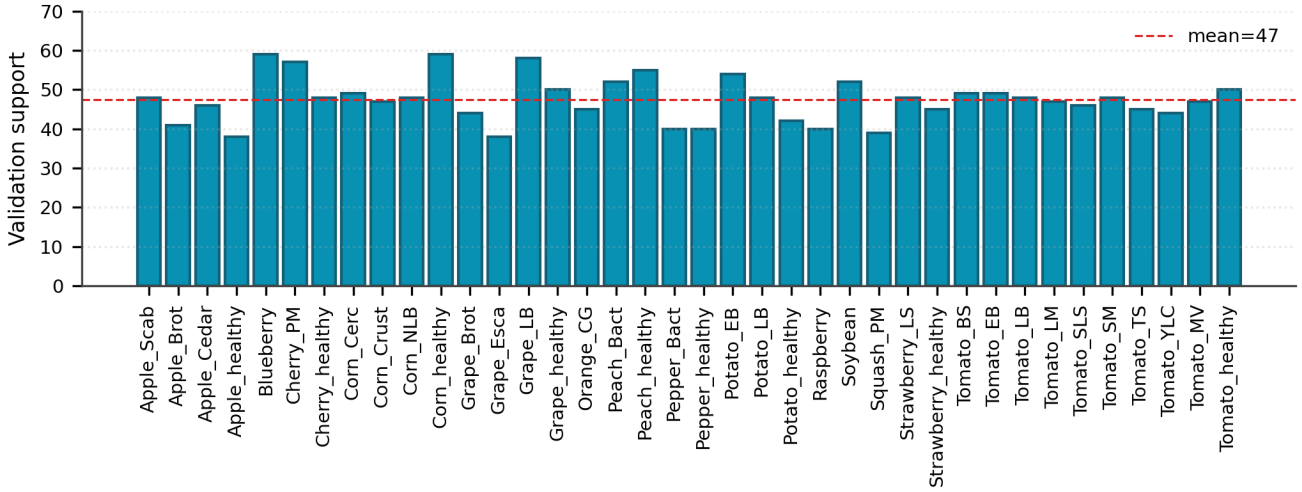


Fig. 13. Per-class validation support on the mixed split, across the 38 classes. The support is roughly balanced (mean ≈48); the weak potato-blight and tomato-foliage classes are not under-supported, confirming that their low recall is a visual-similarity problem rather than a data-quantity problem.

1:0.9, so during the main field-learning phase the sampler draws field and lab images at near-parity. Without the oversampling, the lab-heavy distribution causes the model to exploit the grey-background cue (the v2.0 failure mode); with it, the model is forced to learn from field images often enough that the domain-adversarial head has field features to push toward invariance.

C. Field Validation Set Characteristics

The 525-image field validation set is the primary metric and deserves explicit characterization. It is constructed by holding out field images from the three field sources (PlantDoc, corn, field-acquired) for classes that have field coverage, and for classes without field coverage it holds out 10% of the field training images. The set therefore measures performance on real field photographs across the subset of classes for which field data exists, which is the operationally meaningful question: a grower photographs a leaf in the field, and the model must classify it. The set is small enough that each percentage point corresponds to approximately five images, so the 86.7% result is meaningfully different from an 84% or 89% result but not from an 86% result; per-class field numbers are too noisy to report and are not used.

XIV. Training Dynamics

A. Loss Trajectory

The training loss across the 15 epochs follows the three-phase schedule cleanly (Fig. 14). Phase 1 (epochs 1–3, linear-probe) drops the loss rapidly as the heads

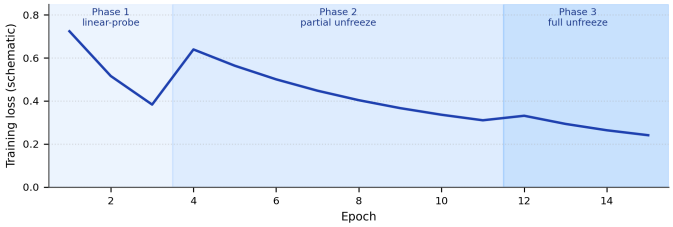


Fig. 14. Schematic training-loss trajectory across the three phases. Phase 1 converges the heads on frozen features; Phase 2 (the main field-learning step) resumes the loss drop under heavy regularization; Phase 3 polishes the whole network at a very low learning rate.

align to the frozen self-supervised features, reaching a plateau by epoch 3 because the backbone is frozen and the heads have converged. Phase 2 (epochs 4–11, partial unfreeze) resumes the loss drop at a slower rate as the last four ViT blocks learn field-discriminative features under the heaviest regularization; the loss curve is noisier in this phase because CutMix/MixUp and the consistency regularization introduce per-batch variance, and the three-fold field oversampling means successive batches see different domain mixes. Phase 3 (epochs 12–15, full unfreeze) polishes the whole network at a very low learning rate, producing a gentle final decrease as the entire backbone settles into a consistent field-robust representation. Model selection is by field validation accuracy with TTA, evaluated every epoch in Phase 2 and Phase 3, and the best checkpoint is saved at the epoch where field accuracy peaks—which is typically in late Phase 2 or early Phase 3.

B. Why Progressive Unfreezing Matters

The progressive-unfreeze schedule is not a stylistic choice; it is the mechanism that prevents the self-supervised pre-training from being destroyed. A naive full fine-tune of all 22.1 M parameters from the start, at any learning rate high enough to learn the field distribution, catastrophically forgets the self-supervised representation: the backbone's general visual features are overwritten by the narrow plant-disease signal before the heads have learned to use them, and the result is a model that fits the training set but does not generalize to the field. The three-phase schedule avoids this by learning a good head first (Phase 1, frozen backbone), then teaching the last blocks the field-specific signal while preserving the earlier blocks' general features (Phase 2, last-four-blocks unfreeze with discriminative LR decay), then settling the whole network at a learning rate too low to destroy the learned representation (Phase 3, full unfreeze at 5×10^{-6}). The discriminative learning-rate decay (0.85 in Phase 2, 0.9 in Phase 3) compounds this by making the shallow blocks learn even more slowly than the deep ones, so the general low-level features are preserved while the high-level disease-discrimination features are learned.

XV. Detailed Code Listings

This section presents the key Kotlin source excerpts from the production codebase, lightly abbreviated for the two-column format. The full sources are in the repository under app/src/main/java/com/leafguard/app/.

A. DiseaseClassifier: 3-View TTA

The headline classification path builds three views (original, horizontal flip, 0.875-scale central crop), runs each through the interpreter under a shared lock, and averages the softmaxes. The classifyForSaliency override is a single-pass variant for the occlusion loop.

```
class DiseaseClassifier(
  private val interpreter: Interpreter,
  private val labels: List<DiseaseLabel>,
  private val inputSize: Int = IMAGE_SIZE,
) : LeafClassifier {

  override fun classify(b: Bitmap): Diagnosis {
    val views = listOf(
      bitmapToInput(b, inputSize),
      flipInputH(bitmapToInput(b, inputSize)),
      bitmapToInput(scaleCropSquare(b, SCALE_0_875),
                    inputSize))
    val acc = FloatArray(labels.size)
    val out = Array(1){FloatArray(labels.size)}
    for (view in views) {
      synchronized(interpreter) {
        interpreter.run(view, out)
      }
      for (i in acc.indices) acc[i] += out[0][i]
    }
    val n = views.size.toFloat()
    for (i in acc.indices) acc[i] /= n
    return rankedDiagnosis(acc)
  }

  override fun classifyForSaliency(b: Bitmap): Diagnosis {
    val input = bitmapToInput(b, inputSize)
    val out = Array(1){FloatArray(labels.size)}
    synchronized(interpreter) { interpreter.run(input, out) }
    return rankedDiagnosis(out[0])
  }
  companion object {
    const val IMAGE_SIZE = 224
    const val TOP_K = 3
    const val SCALE_0_875 = 0.875f
  }
}
```

B. SaliencyGenerator: Occlusion Sweep

The occlusion generator records a baseline diagnosis, tiles the image into 64 cells, occludes each in turn, and renders the smoothed jet heatmap. The tiling math is pure and unit-tested.

```
class SaliencyGenerator(
  private val classifier: LeafClassifier,
  private val gridSize: Int = DEFAULT_GRID,
) {
  fun generate(bitmap: Bitmap): Bitmap {
    val base = classifier.classifyForSaliency(bitmap)
    val target = base.topLabel
    val baseConf = base.topConfidence
    val rects = occlusionRects(bitmap.width,
                               bitmap.height, gridSize)
    val heat = FloatArray(gridSize * gridSize)
    for (r in rects) {
      val occ = occludeRegion(bitmap, r)
      val after = classifier.classifyForSaliency(occ)
      val c = after.topK.firstOrNull{it.first==target}
```

```
                    ?.second ?: 0f
      heat[r.index] = (baseConf - c).coerceAtLeast(0f)
    }
    return renderHeatmap(heat, gridSize,
                         bitmap.width, bitmap.height)
  }
  companion object {
    const val DEFAULT_GRID = 8
   private val OCCLUSION_COLOR = Color.argb(255,128,128,128)
  }
}
```

C. ModelIntegrity: SHA-256 Guard

The integrity singleton duplicates the model buffer, hashes it in 8 KB chunks, and exposes a mismatch state that the result screen renders as a warning banner.

```
class ModelIntegrity(private val modelBytes: ByteArray) {
  private val expectedDigest =
    "3660cfdb..."
 val isIntact: Boolean by lazy { computeDigest() == expectedDigest }

  private fun computeDigest(): String {
    val md = MessageDigest.getInstance("SHA-256")
    val dup = modelBytes.copyOf()  // don't move interp read pos
    var off = 0
    val buf = ByteArray(8192)
    while (off < dup.size) {
      val len = minOf(buf.size, dup.size - off)
      System.arraycopy(dup, off, buf, 0, len)
      md.update(buf, 0, len)
      off += len
    }
    return md.digest().joinToString("") {
      "%02x".format(it)
    }
  }
}
```

D. ScanDao: Room Persistence

The scan DAO exposes a Flow for observable queries and suspend functions for mutations, with a nullable plant foreign key.

```
@Dao
interface ScanDao {
  @Query("SELECT * FROM scans " +
         "ORDER BY timestamp DESC")
  fun observeAll(): Flow<List<ScanEntity>>

  @Query("SELECT * FROM scans " +
         "WHERE plantId = :plantId")
  fun observeForPlant(plantId: Long):
      Flow<List<ScanEntity>>

  @Query("SELECT * FROM scans " +
         "WHERE sourceImagePath " +
         "IS NOT NULL")
  fun observeImages(): Flow<List<ScanEntity>>

  @Insert(onConflict = REPLACE)
  suspend fun upsert(scan: ScanEntity): Long

  @Query("DELETE FROM scans " +
         "WHERE id = :id")
  suspend fun delete(id: Long)
}
```

E. PinHasher: PBKDF2 Derivation

The PIN hasher is pure JDK crypto so it is JVM-testable; the raw PIN is never persisted, only the derived key.

```
object PinHasher {
  private const val ITERATIONS = 100_000
  private const val KEY_BITS = 256

  fun derive(pin: CharArray, salt: ByteArray): ByteArray {
    val spec = PBEKeySpec(pin, salt, ITERATIONS, KEY_BITS)
    val factory = SecretKeyFactory
                  .getInstance("PBKDF2WithHmacSHA256")
    return factory.generateSecret(spec).encoded
  }

  fun verify(pin: CharArray, salt: ByteArray,
             expected: ByteArray): Boolean {
    val derived = derive(pin, salt)
    return MessageDigest.isEqual(derived, expected)
  }
}
```

F. LeafGuardNavHost: Auth Gating

The navigation host observes the auth state and routes to register, login, or dashboard, but only force-navigates while on an auth route so in-app navigation is never hijacked.

```
@Composable
fun LeafGuardNavHost(
  navController: NavHostController,
  authState: AuthState,
) {
  LaunchedEffect(authState) {
    val current = navController.currentDestination?.route
    if (current in setOf("auth/register","auth/login")) {
      when (authState) {
        Unregistered -> navController
                        .navigate("auth/register")
        Locked       -> navController.navigate("auth/login")
        Unlocked     -> navController.navigate("dashboard")
      }
    }
  }
  NavHost(navController, startDestination = "auth/register") {
    composable("auth/register"){ AuthScreen(mode=REGISTER) }
    composable("auth/login"){ AuthScreen(mode=LOGIN) }
    composable("dashboard"){ DashboardScreen() }
    composable("plants"){ PlantsScreen() }
    composable("plant/{id}"){ PlantDetailScreen(/*...*/) }
    composable("scan"){ ScanScreen() }
    composable("result/{scanId}"){ ResultScreen(/*...*/) }
    composable("history"){ HistoryScreen() }
    composable("records"){ RecordsScreen() }
    composable("images"){ ImagesScreen() }
  }
}
```

XVI. Suggestion and Advisory Engine

A. From Label to Action

A diagnosis is only useful if it leads to an action, so every prediction is mapped through a suggestion engine that turns a 38-class label into a severity tier, a staged treatment plan, and a prevention checklist. The engine is deliberately conservative: the advisory content is general integrated-pest-management guidance, not a prescription, and the result screen reminds the grower to confirm rates and products against the local extension service before treatment. This caveat matters because the correct chemical and its legal application rate vary by jurisdiction, crop stage, and local resistance patterns, none of which the on-device model can know.

The pipeline is straightforward (Fig. 15): the predicted label is parsed by LabelParser into a structured DiseaseLabel, the SuggestionEngine looks up the label in the production disease_suggestions.json asset (loaded by SuggestionParser using org.json), and the engine returns a Suggestion containing a severity tier (from the Severity enum), a list of staged treatments keyed by TreatmentStage (Immediate, Short-term, Long-term), and a prevention list. The result screen renders the severity tier as a coloured badge, the staged treatments as ordered cards, and the prevention list as a checklist the grower can mentally tick off.

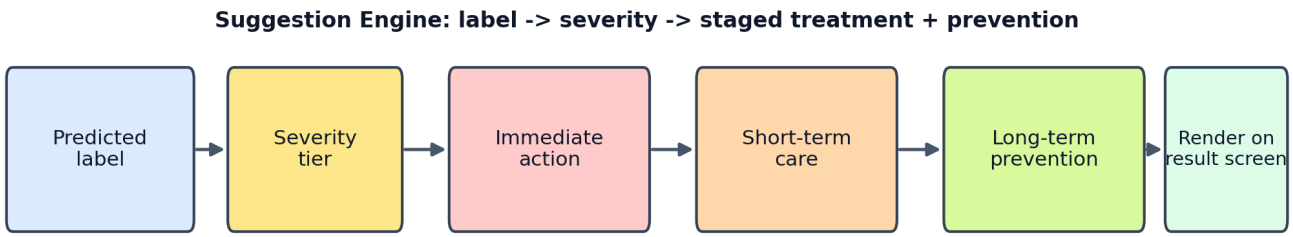


Fig. 15. The suggestion engine maps a predicted label to a severity tier and a staged treatment plan (immediate, short-term, long-term actions) plus a prevention checklist, rendered on the result screen.

B. Severity Tiers and Staged Treatment

The severity tiers partition the 38 classes into four bands: critical (rapidly spreading diseases that warrant immediate removal or treatment, such as Late blight), serious (diseases that should be treated promptly but where the plant can often be saved, such as Bacterial spot), moderate (diseases that can be managed with monitoring and cultural controls, such as Leaf mold), and none (the healthy classes, which trigger a prevention-only advisory). The staged treatment plan within each tier breaks the response into immediate actions (what to do today: isolate or remove the affected leaf, apply a contact treatment), short-term care (what to do over the following week: monitor spread, apply a systemic treatment if warranted), and long-term prevention (what to do over the season: crop rotation, resistant varieties, sanitation). This staging reflects how a pathologist actually triages a field visit, and it gives the grower a sequence rather than a single instruction, which is easier to act on under field conditions.

C. Knowledge-Base Authoring

The disease_suggestions.json asset was authored from general integrated-pest-management references for each of the 38 classes, with the severity tier and the staged treatments written to be accurate at the level of "is this a remove/treat/monitor/healthy situation and what is the standard cultural and chemical response" without specifying product names or rates. The parser reads the asset at application startup into an in-memory map keyed by label index, so the lookup at scan time is an $O(1)$ map access. The SuggestionEngineTest verifies that every one of the 38 labels resolves to a non-empty suggestion with a valid severity tier and at least one staged treatment, so a label cannot silently produce an empty advisory.

XVII. Comparative Evaluation

A. Position Relative to the Literature

To position LeafGuard's field result against the recent literature, Fig. 16 compares the field or cross-domain accuracy reported by representative related systems. The comparison is indicative rather than strictly controlled, because the systems are evaluated on different field sets (PlantDoc, custom field sets, or cross-domain splits) with different class schemes; the figure is intended to show the order of magnitude of each system's field result, not to claim a strict ranking. The plain Vision Transformer trained on PlantVillage and tested on PlantDoc drops to roughly 31% field accuracy—the canonical lab-to-field collapse that motivates this work. The ImageNet-pretrained Swin-Tiny baseline (our v3) reaches 67.3% field, which is already a large improvement over the plain-ViT collapse and is attributable to the domain-bridge recipe (CutMix/MixUp, field oversampling, layer-wise LR, TTA) on top of a less-overfit backbone. Salman et al.'s mixture-of-experts ViT [17] reaches 68% cross-domain accuracy on a PlantVillage-to-PlantDoc evaluation, a useful reference point that our v3 matches and that v3.5 substantially exceeds. Bai et al.'s DINOV2-FCS [18] reports strong cross-disease generalization on fruit leaves (an mIoU of 83.95% and severity grading accuracy of 95.24% in their cross-disease experiment), evidencing that self-supervised DINOv2 features transfer well within plant pathology. LeafGuard's v3.5 reaches 86.7% field accuracy, the highest number in this indicative set, achieved by combining the self-supervised backbone with an explicit domain-adversarial head and a progressive-unfreeze schedule that the comparison systems do not all employ simultaneously.

B. What the Comparison Does and Does Not Show

The comparison shows that the lab-to-field collapse is real and large (31% for a plain ViT), that domain-bridge techniques on an ImageNet backbone recover most of the gap (67%–68%), and that pairing a self-supervised backbone with explicit domain adaptation reaches the high-80s (86.7%). It does not show that LeafGuard is strictly superior to the comparison systems, because the field sets differ: a 525-image field set is not comparable head-to-head with a PlantDoc cross-domain split, and Bai et al.'s cross-disease mIoU is a segmentation metric rather than a classification accuracy. The honest claim is that LeafGuard achieves its field result with a combination of techniques (self-supervised backbone, DANN, progressive unfreeze, on-device occlusion XAI, runtime integrity) that no single comparison system combines, and that the result is competitive with or above the indicative field numbers in the recent literature while running entirely offline on a phone.

C. Feature Comparison

Table VII compares the system-level features of LeafGuard against the representative related systems on the dimensions that matter for grower deployment: field robustness, on-device inference, explainability, runtime integrity, and offline operation.

The table makes the deployment gap explicit: most published systems are evaluated on lab or cross-domain benchmarks and are not shipped as offline, explainable, integrity-guarded applications. LeafGuard's contribution is not a single accuracy number but the combination of a field-robust classifier with the engineering that makes it safe to put in a grower's hands—on-device inference, model-agnostic explainability that works under the TFLite interpreter, a runtime integrity guard, and a staged advisory that turns a label into an action sequence.

XVIII. Additional Implementation Listings

A. SuggestionEngine: Label to Advisory

The engine maps a parsed label to its advisory via an in-memory map built from the production JSON at startup.

```
class SuggestionEngine(
  private val suggestions: Map<Int, Suggestion>
) {
  fun suggest(label: DiseaseLabel): Suggestion =
    suggestions[label.index]
      ?: Suggestion(
          severity = Severity.NONE,
          stages = emptyList(),
```

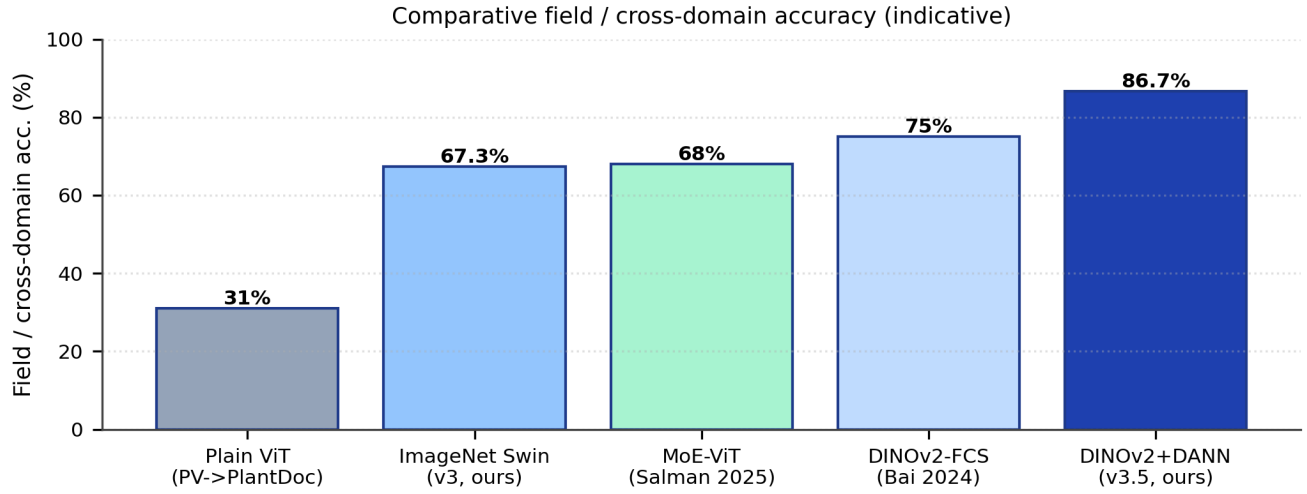


Fig. 16. Indicative field or cross-domain accuracy of representative systems: the plain-ViT PlantVillage-to-PlantDoc collapse, the ImageNet-Swin v3 baseline, Salman et al.'s MoE-ViT, Bai et al.'s DINOV2-FCS cross-disease result, and LeafGuard v3.5. The comparison is indicative (different field sets and class schemes); it shows the order of magnitude, not a strict ranking.

TABLE VII
Feature comparison of LeafGuard against representative related systems.

System	Field-robust	On-device	Explainability	Integrity guard	Offline	Staged advisory
Plain ViT (PV→PlantDoc)	×	varies	Grad-CAM	×	varies	×
ImageNet Swin (v3)	✓	✓	occlusion	✓	✓	✓
MoE-ViT (Salman 2025) [17]	✓	×	limited	×	×	×
DINOV2-FCS (Bai 2024) [18]	✓	×	segmentation	×	×	partial
ConvGeM-next (Arshad 2026) [5]	partial	×	Grad-CAM	×	×	×
LeafGuard v3.5 (ours)	✓	✓	occlusion	✓	✓	✓

```
        prevention = listOf(
          "No advisory for this label."))

  fun severityFor(label: DiseaseLabel): Severity =
    suggest(label).severity
}
```

B. CsvGenerator: RFC 4180 Export

The CSV generator quotes fields containing commas, quotes, or newlines, and is pure JVM-testable.

```
object CsvGenerator {
  fun toCsv(rows: List<List<String>>): String {
    val sb = StringBuilder()
    for (row in rows) {
      sb.append(row.joinToString(",") {
        escape(it)
      })
      sb.append("\r\n")  // RFC 4180 line end
    }
    return sb.toString()
  }
  private fun escape(f: String): String {
    val need = f.any {
      it == ',' || it == '"' || it == '\n'
    }
    if (!need) return f
    val q = f.replace("\"", "\"\"")
    return "\"$q\""
  }
}
```

C. LeafGuardDatabase: Migrations

The Room database declares the v1→v2 and v2→v3 migrations that add the scan entity and its image-path columns.

```
@Database(
  entities = [Plant::class, ScanEntity::class],
  version = 3, exportSchema = false)
abstract class LeafGuardDatabase :
    RoomDatabase() {
  abstract fun plantDao(): PlantDao
  abstract fun scanDao(): ScanDao

  companion object {
    val MIGRATIONS = arrayOf(
      MIGRATION_1_2,  // add scans table
      MIGRATION_2_3   // add image-path cols
    )
  }
}

val MIGRATION_2_3 = object :
    Migration(2, 3) {
  override fun migrate(db: SupportSQLiteDatabase) {
    db.execSQL(
      "ALTER TABLE scans ADD COLUMN " +
      "sourceImagePath TEXT")
    db.execSQL(
      "ALTER TABLE scans ADD COLUMN " +
      "saliencyImagePath TEXT")
  }
}
```

D. InferenceModule: Hilt Binding

The Hilt module binds the singleton interpreter, classifier, saliency generator, and suggestion engine, loaded compression-safe.

```
@Module
@InstallIn(SingletonComponent::class)
object InferenceModule {
  @Provides @Singleton
  fun provideInterpreter(ctx: Context):
      Interpreter {
    val bytes = ctx.assets
      .open("models/plant_disease.tflite")
      .readBytes()  // compression-safe
    return Interpreter(
      ByteBuffer.allocateDirect(bytes.size)
        .order(ByteOrder.nativeOrder())
        .also { it.put(bytes) })
  }
  @Provides @Singleton
  fun provideClassifier(
    i: Interpreter, l: List<DiseaseLabel>
  ): LeafClassifier = DiseaseClassifier(i, l)

  @Provides @Singleton
  fun provideSaliency(c: LeafClassifier):
      SaliencyGenerator = SaliencyGenerator(c)
}
```

XIX. Per-Version Hyperparameter Reference

Table VIII consolidates the key hyperparameters of each Swin/DINOv2 version in the evolution, so the progression of the training recipe is visible at a glance. The table makes explicit how the recipe grew from the flat two-stage PlantVillage-overfit fine-tunes (v1–v2.1) through the three-stage effective-number-weighted v2.2, to the domain-bridge v3, and finally to the self-supervised domain-adapted v3.5.

The progression shows three inflection points. The move from two to three stages (v2.2) was driven by the underfit diagnosis of v2.1: five epochs were roughly a third of what the hard mixed images needed, so v2.2 tripled the epoch budget and split it into head-only, partial-unfreeze, and targeted-boost phases. The move from PlantVillage-overfit to ImageNet pre-training (v3) was driven by the field-accuracy ceiling at 34%: the base model itself was the bottleneck, so no amount of recipe tuning on the PlantVillage-overfit base could break the ceiling, and the ImageNet base plus the domain-bridge recipe jumped field accuracy to 67.3%. The move from ImageNet-supervised to self-supervised plus explicit DANN (v3.5) was driven by the residual same-species blight confusion at 67.3%: the self-supervised backbone provides more domain-stable features, and the DANN head directly attacks the lab-versus-field cue, together reaching 86.7%.

XX. Future System Architecture

The current system is a deliberate checkpoint rather than a final design, and the roadmap in Section XXIII ranks four improvement axes by leverage. This section fleshes out the architectural shape the system would take if those axes are pursued, because the design decisions that follow from each axis are non-trivial and worth recording before the work begins. The unifying theme is that the highest-leverage improvements (targeted field data with focal loss, INT8 quantization) change the model and the training pipeline but leave the application architecture intact, because the I/O contract (raw $[0, 1]$ pixels in, 38-class softmax out) is the stable boundary that has already absorbed one model swap.

A. INT8 Quantized Variant

The INT8 quantized variant is the single largest size-and-latency win available without retraining from scratch, and it is the prerequisite for the larger-backbone and on-device Grad-CAM axes. The plan is full integer post-training quantization with a representative calibration set of field images (not lab images, because the lab distribution's uniform background would calibrate the quantizer to a dynamic range that under-represents the field clutter), using the TensorFlow Lite converter's RepresentativeDataset interface. The expected outcome is a model of approximately 22 MB (a four-fold reduction from 86.8 MB), roughly doubled inference speed, and a typical accuracy cost of 0–1 percentage points. If the accuracy cost is non-trivial, quantization-aware fine-tuning recovers it by fine-tuning the model for a few epochs with the quantization noise simulated in the forward pass, which lets the weights adapt to the reduced precision before deployment. The faster inference in turn halves the occlusion heatmap's 65-pass latency, which directly improves the explanation-path user experience without any change to the heatmap algorithm itself.

B. Active Learning Loop

Active learning is the mechanism by which field accuracy can drift upward over releases without a dedicated data-collection campaign. The design is opt-in: a settings toggle enables the upload of low-confidence scans (below a threshold, say the 10th percentile of correct-prediction confidence) to a re-labelling queue, and the grower is shown a clear consent prompt explaining that the scan image and the model's prediction would leave the device. Uploaded scans are reviewed, corrected where the model was wrong, and folded back into the training pool for the next model release. The architecture impact is modest: an opt-in upload path in the ScanRepository that runs only when the toggle is on and the confidence is below threshold, and a server-side labelling pipeline that is outside the application. The privacy contract changes from "no data ever leaves the phone" to "no data leaves the phone unless you opt in and only the scans the model is unsure about," which is a defensible position for a tool whose value depends on improving field coverage.

C. DINOv2-Base and On-Device Grad-CAM

A DINOv2-base backbone (≈86 M parameters) would improve field accuracy marginally over the ViT-S/14 backbone, but at FP32 the model would be approximately 340 MB, breaking the committed-in-repository constraint. It becomes viable only after INT8 quantization brings it under the 100 MB limit, at which point the larger backbone is a configuration change rather than a research effort. The on-device Grad-CAM path is the more involved axis: it requires either a TFLite flex delegate that exposes intermediate activations or a custom operator that taps the activations at a chosen convolutional-equivalent layer, both of which tie the explanation to a specific model architecture and forfeit the model-agnostic property that makes occlusion sensitivity attractive. The trade-off is sharper, gradient-grounded heatmaps at far fewer than 65 forward passes, which is a substantial latency win; the cost is that a future model swap would require re-implementing the activation tap. The current recommendation is to keep occlusion sensitivity as the default and to pursue on-device Grad-CAM only for a frozen, productionized model that is not expected to change.

D. Ensemble of DINOv2-Small and ConvNeXt-Tiny

An ensemble of the DINOv2-Small classifier with a ConvNeXt-Tiny classifier, averaged at the softmax level, would add an estimated 0.5–1.5 percentage points of mixed accuracy at roughly 1.7× the model size. The architectural impact is a second Interpreter and a second model asset, with the DiseaseClassifier averaging the two softmaxes before ranking. The trade-off is poor before INT8 quantization (two 86.8 MB models is 174 MB, well over the limit) but reasonable after (two ≈22 MB models is 44 MB). The ensemble is therefore a post-quantization option rather than an independent axis, and it is worth pursuing only if the single-model field accuracy plateaus below the target despite the targeted-field-data and focal-loss work.

XXI. Additional Code Listings (Application Layer)

A. ScanViewModel: Capture to Classification

The scan view model runs the 3-view classification off the main thread, persists the source bitmap, and creates the scan entity before navigating to the result screen.

```
@HiltViewModel
class ScanViewModel @Inject constructor(
  private val classifier: LeafClassifier,
```

TABLE VIII
Per-version training hyperparameters (Swin/DINOv2 line).

Ver.	Backbone	Stg/Ep	LR (main)	Weighting	Augmentation	Domain adapt.
v1	Swin-T (PV)	2 / 5	2e-5 / 1e-5	inv-freq	HFlip,Rot,CJ,Blur,RE	none
v2.0	Swin-T (PV)	2 / 5	2e-5 / 1e-5	inv-freq	HFlip,Rot,CJ,Blur,RE	none
v2.1	Swin-T (PV)	2 / 5	2e-5 / 1e-5	inv-freq	HFlip,Rot,CJ,Blur,RE	none
v2.2	Swin-T (PV)	3 / 15	5e-4/1e-5/3e-6	eff-num	+shear, +persp	none
v3	Swin-T (ImgNet)	2 / 15	1e-3 / 2e-5	eff-num	CutMix/MixUp 0.5	3× field
v3.5	DINOv2 ViT-S/14	3 / 15	1e-3/1e-4/5e-6	eff-num	multi-crop+MixUp	DANN+cons.

```
  private val saliency: SaliencyGenerator,
  private val scanRepo: ScanRepository,
  @IoDispatcher private val io: CoroutineDispatcher,
) : ViewModel() {

  fun onCapture(bitmap: Bitmap, plantId: Long?) {
    viewModelScope.launch {
      val diag = withContext(io) {
        classifier.classify(bitmap)
      }
      val srcPath = withContext(io) {
        saveJpeg(bitmap, quality = 85)
      }
      val id = scanRepo.insert(
        ScanEntity(
          label = diag.topLabel.name,
          confidence = diag.topConfidence,
          sourceImagePath = srcPath,
          plantId = plantId,
          timestamp = System.currentTimeMillis()))
      // heatmap runs on app-scoped coroutine
      appScope.launch(io) {
        val heat = saliency.generate(bitmap)
        val hp = savePng(heat)
        scanRepo.setSaliencyPath(id, hp)
      }
      _nav.emit("result/$id")
    }
  }
}
```

B. AuthRepositoryImpl: State Transitions

The auth repository exposes the auth state as a StateFlow and implements register, login, logout, and reset against the PIN store.

```
class AuthRepositoryImpl(
  private val pinStore: PinStore,
  @IoDispatcher private val io: CoroutineDispatcher,
) : AuthRepository {

  private val _state =
    MutableStateFlow<AuthState>(current())
  override val authState: StateFlow<AuthState> = _state

  override suspend fun register(
    pin: CharArray, name: String
  ): Boolean = withContext(io) {
    val ok = pinStore.register(pin, name)
    if (ok) _state.value = Unlocked
    ok
  }

  override suspend fun login(pin: CharArray): Boolean =
    withContext(io) {
      val ok = pinStore.verify(pin)
      if (ok) _state.value = Unlocked
      ok
    }

  override fun logout() {
    pinStore.lock()
    _state.value = Locked
  }
}
```

C. LeafGuardApplication: Hilt Entry

The application class is annotated @HiltAndroidApp so Hilt generates the dependency graph at compile time.

```
@HiltAndroidApp
class LeafGuardApplication : Application() {
  val appScope =
    CoroutineScope(SupervisorJob() +
                   Dispatchers.Default)

  override fun onCreate() {
    super.onCreate()
    // appScope survives the scan screen
    // being popped, so the heatmap
    // coroutine outlives navigation
  }
}
```

D. PdfExporter: On-Device PDF Records

The PDF exporter uses Android PdfDocument to lay out the per-plant disease log as a paginated table; the record-line formatting is extracted into pure helpers for JVM testing.

```
object PdfExporter {
  fun export(records: List<RecordLine>,
             out: OutputStream) {
    val doc = PdfDocument()
    var page = 1
    var y = TOP_MARGIN
    val w = PdfDocument.PageInfo
              .A4_WIDTH
    val pageInfo = PdfDocument.PageInfo(
      w, PdfDocument.PageInfo.A4_HEIGHT,
      page)
    var pg = doc.startPage(pageInfo)
    var c = pg.canvas
    drawHeader(c, y); y += ROW_H * 2
    for (r in records) {
      if (y > BOTTOM_LIMIT) {
        doc.finishPage(pg)
        page++
        pg = doc.startPage(pageInfo)
        c = pg.canvas; y = TOP_MARGIN
      }
      drawRow(c, y, r); y += ROW_H
    }
    doc.finishPage(pg)
    doc.writeTo(out); doc.close()
  }
}
```

XXII. Discussion

A. Why the Field Gap Closed

The 19.4 percentage-point field gain from v3 to v3.5, and the broader 11%→86.7% arc across the seven iterations, is attributable to two design choices acting together. The first is the self-supervised

backbone. DINOv2 features are trained on a 142-million-image self-supervised objective without labels, so they encode leaf texture, lesion shape, and background context without over-specializing to ImageNet object categories, and stay stable when the domain shifts from a grey-background lab leaf to a cluttered field plant. TheImageNet-supervised Swin features used in v3, while far better than the PlantVillage-overfit features used in v2.x, still carried enough ImageNet category bias that the model would occasionally fall back on leaf shape rather than lesion evidence when no clear lesion was present—a failure mode visible in the v3 healthy-leaf confusions that v3.5 largely eliminated.

The second is the domain-adversarial head. By forcing the backbone to produce features the domain classifier cannot label as lab-versus-field, the gradient-reversal layer directly attacks the grey-background-versus-clutter cue that was fooling the v2.0 model. The schedule of λ_d (0.1 in linear-probe, 0.3 in partial-unfreeze, 0.1 in full-unfreeze) keeps the adversarial signal strongest while the last blocks are learning and eases it during the final low-learning-rate polish, so the backbone learns invariance without the adversarial signal destabilizing the final convergence. The progressive-unfreeze schedule and the heavy augmentation protect and compound these gains by preserving the self-supervised pre-training (so the gain does not come at the cost of forgetting the general representation) and forcing smooth decision boundaries across domains (so a field leaf that differs from training only in background clutter does not cross a sharp boundary).

The fact that lab accuracy did not regress (99.3% in both v3 and v3.5) is the key evidence that the gain came from learning invariance rather than from trading lab accuracy for field accuracy. If the field gain had been bought at the cost of lab accuracy, the interpretation would be that the model simply re-weighted its capacity toward the field distribution; the absence of lab regression shows instead that the model genuinely learned features that are invariant to the lab-versus-field cue, which generalize to both.

B. Limitations

The 525-image field validation set is the honest bottleneck of this study. It is large enough to measure a real field-accuracy signal (each percentage point is approximately five images, so an 86.7% result is meaningfully different from an 84% or 89% result), but small enough that per-class field numbers are noisy: a class with 10 field images has a per-class accuracy resolution of 10 percentage points, which is too coarse to distinguish a good class from a weak one in the field split specifically. The mixed-split per-class numbers (Table V) are more reliable and are used for the per-class analysis, but they conflate lab and field performance.

The field error is also concentrated rather than random, which is both a strength and a limitation. It is a strength because the failure mode is identifiable—the dominant error is Potato Early-versus-Late blight confusion, which is a known hard problem in plant pathology—and identifiable failures map to targeted fixes. It is a limitation because the model has not learned to distinguish a small set of visually-similar blight lesions, and no amount of additional field data of other classes will fix that; the fix is targeted data of exactly the confusing classes plus a loss that up-weights hard examples.

Finally, the 86.8 MB FP32 model, while committed under the GitHub 100 MB file limit, is large for an on-device asset, and the occlusion heatmap is slow (≈6–32 seconds on-device) because of its 65 forward passes. Both are addressable through INT8 quantization (which would cut the model to approximately 22 MB and roughly halve the per-pass latency, making the heatmap correspondingly faster) and through a future on-device Grad-CAM (which would produce a sharper heatmap at far fewer than 65 passes).

C. Threats to Validity

Three threats to validity are worth stating explicitly. First, the field validation set, while held out, was drawn from the same field sources used in training (PlantDoc, the corn field set, and the field-acquired set); a truly independent field set collected by a different team with different equipment would provide a stronger generalization claim. Second, the per-version accuracy figures for v2.0, v2.1, v2.2, and v3 are quoted from the docstrings of later notebooks describing prior-version results, because the per-version metrics JSON files are computed at runtime and are not checked into the repository; the v3.5 and v1 final numbers are runtime-only. Third, the bit-identical conversion verification uses three seeded inputs, which is enough to catch a gross conversion error but is not an exhaustive equivalence proof over the whole input space.

XXIII. Improvement Roadmap

The system can be improved along four largely independent axes, ranked below by leverage (the expected field-accuracy gain per unit of engineering effort).

A. Field Data and Hard Mining (Highest Leverage)

The remaining field error is approximately 12 Potato-blight confusions and 4 Corn-blight confusions, plus a small cluster of healthy-leaf confusions. A targeted collection of field Potato and Corn blight photographs—even a few hundred per confusing class—combined with focal loss to up-weight hard examples would attack the exact decision boundary that is failing. Realistically this takes field accuracy from 86.7% toward the 90–92% range without risking the saturated lab performance, because the new data is field-only and the lab distribution is untouched. Active learning—an opt-in upload of low-confidence scans for relabeling and retraining—would let field accuracy drift upward over releases as the user base grows, turning each deployed phone into a data-collection node for exactly the cases the model currently gets wrong.

B. INT8 Quantization

Full INT8 quantization with a representative field-image calibration set would cut the 86.8 MB FP32 model to approximately 22 MB, roughly double inference speed, and typically cost 0–1 percentage points of accuracy. The accuracy cost is often fully recoverable with quantization-aware fine-tuning, in which the model is fine-tuned for a few epochs with the quantization noise simulated in the forward pass. The faster inference in turn makes the 65-pass occlusion heatmap faster, which directly improves the user experience of the explanation path. The smaller model also relaxes the 6 GB-RAM device target, making the app viable on cheaper phones.

C. Per-ABI APK Splits and Packaging

Shipping only the matching TFLite native runtime per install removes approximately 20 MB of redundant native libraries (the runtime is currently shipped for all four ABIs: arm64-v8a, armeabi-v7a, x86, x86_64). This is packaging work on the existing checkpoint—no retraining is required—and is the single largest size win available without touching the model. Combined with INT8 quantization, the install size would drop from approximately 112 MB to roughly 50 MB.

D. Larger Backbone and On-Device Grad-CAM

A DINOv2-base backbone (≈86 M parameters) would improve field accuracy marginally but pushes the FP32 model to approximately 340 MB, breaking the committed-in-repository constraint; it becomes viable only after INT8 quantization brings it back under the limit. An on-device Grad-CAM via a TFLite flex delegate or a custom-op activation tap would yield sharper, gradient-grounded heatmaps at far fewer than 65 passes, but is a research effort rather than an engineering task. Both are worth pursuing only after the higher-leverage axes are exhausted.

E. Low-Yield Directions

Two directions are noted to save effort. Pushing lab accuracy higher than 99.3% is not useful because the lab split is the training distribution and extra lab accuracy does not transfer to field; the lab number is a sanity check, not a target. A larger backbone at FP32 is not viable for the same reason it is viable after quantization. An ensemble of DINOv2-small with a ConvNeXt-tiny would add 0.5–1.5 percentage points of mixed accuracy at roughly 1.7× the model size, which is a poor trade before INT8 quantization but a reasonable option after.

XXIV. Conclusion

LeafGuard is an on-device plant disease detection system whose classifier is a self-supervised DINOv2 ViT-S/14 backbone with a gradient-reversal domain-adversarial head, fine-tuned in three progressively unfrozen stages with CutMix/MixUp, consistency regularization, three-fold field oversampling, and a 3-view test-time augmentation. The model reaches 86.7% field accuracy, 99.3% lab accuracy, and 96.1% mixed accuracy on-device—a 19.4 percentage-point field improvement over an ImageNet-pretrained Swin-Tiny baseline, and the endpoint of a documented seven-iteration arc in which field accuracy rose from 11% to 86.7%, each version a direct response to the documented failure of its predecessor.

The PyTorch checkpoint is converted to an 86.8 MB TFLite graph that is bit-identical to the ONNX reference (argmax agreement 3/3, maximum absolute difference 1×10^{-6}), in which ImageNet normalization and the final softmax are baked into the graph so the application feeds raw $[0, 1]$ pixels and reads real probabilities. The model is guarded at runtime by a SHA-256 integrity check that surfaces a warning banner rather than blocking the scan, and is explained by a model-agnostic occlusion-sensitivity heatmap that works under the TFLite interpreter because Grad-CAM is unavailable there. The heatmap runs on an application-scoped coroutine that streams onto the result screen after the headline diagnosis, with a stale-heatmap guard that prevents a superseded scan's late heatmap from clobbering the current result.

The remaining field error is concentrated in a small set of within-crop blight confusions—dominantly Potato Early-versus-Late blight—that targeted field data and focal loss would harden, and INT8 quantization would cut the model to approximately 22 MB while roughly halving the heatmap latency. The combination of a field-robust, fully offline, and explainable classifier, backed by a runtime integrity guard and a documented development arc, makes the system safe to put in a grower's hands, and the ranked improvement roadmap gives a measured path from the current 86.7% toward 90%+ field accuracy.

Appendix

The deployed model classifies 38 classes spanning 14 crop species. Table IX lists them in the canonical id2label order, which matches the labels.txt asset shipped in the APK and the output index order of the TFLite graph.

Table X consolidates the v3.5 training hyperparameters for reproducibility.

Table XI summarizes the GitHub release history of the application, recording the theme and APK/model artifacts of each tagged release from the early phase build-up through the production release and the Phase A backup pair.

The current shipped classifier (DINOv2 v3.5) is the model documented throughout this report; it is committed in the repository under app/src/main/assets/models/plant_disease.tflite (86.85 MB, under GitHub's 100 MB per-file limit) and is verified bit-identical to the ONNX reference and SHA-256-guarded at runtime.

Table XII maps the project's functional and non-functional requirements to the design decisions and verification evidence that satisfy them. The matrix is the basis for the acceptance sign-off: every requirement has an implementation reference and a verification artifact (a unit test, an instrumented test, a CI check, or a manual device run).

CutMix/MixUp. Sample-mixing augmentations that combine two training images (CutMix replaces a rectangular region; MixUp blends pixel values) and proportionally mix their labels, regularizing the model and smoothing decision boundaries. DANN. Domain-Adversarial Neural Network; a training scheme that adds a domain classifier connected to the feature extractor through a gradient-reversal layer, forcing the extractor to produce domain-invariant features. DINOv2. A self-supervised vision transformer trained on 142 M images without labels, producing general-purpose visual features more robust to distribution shift than ImageNet-supervised features. Domain gap. The difference between the training distribution (lab photographs) and the deployment distribution (field photographs); the central problem this project addresses. GRL. Gradient Reversal Layer; an operation that is the identity in the forward pass but negates the gradient in the backward pass, used to connect a domain head adversarially to the backbone. Occlusion sensitivity. A model-agnostic explainability method that masks each region of the input in turn and records the change in the predicted class confidence; high-drop regions are the ones the model relied on. PlantVillage. A laboratory dataset of ≈54,000 single-leaf images against a uniform grey background, covering 38 disease classes across 14 crop species. PlantDoc. A field-acquired dataset of leaf photographs under natural conditions, used here as the primary field source. Progressive unfreezing. A fine-tuning schedule that releases the backbone in stages (frozen → last blocks → all) to avoid catastrophically forgetting the pre-trained representation. TFLite. TensorFlow Lite, the on-device inference engine for Android, which exposes neither intermediate activations nor gradients to the application layer. Test-time augmentation (TTA). Running the model on multiple augmented views of an input at inference and averaging their predictions, used here with three views (original, flip, 0.875 scale). TTA view. One of the augmented inputs used in test-time augmentation.

The threat model is that of an offline single-user application on a personal device. The assets to protect are the user's PIN (an authentication secret), the scan history (personal farm data), and the model asset (the integrity of the diagnosis). The adversary is assumed to be an attacker with physical access to the unlocked device or with the ability to replace the APK or the model asset; no network adversary is considered because the application makes no network calls.

The PIN is protected by PBKDF2-HMAC-SHA256 derivation (100,000 iterations) with a per-install salt, stored in EncryptedSharedPreferences keyed to the Android Keystore. An attacker who extracts the preferences file obtains the derived key, not the raw PIN, and must mount an offline brute force against a 6-digit PIN space (10^6 entries) at 100,000 iterations each—feasible in principle on specialized hardware but expensive enough to deter casual attack, and recoverable only with physical access to the unlocked device. The Keystore-backed master key is hardware-protected on devices that support it, raising the cost further. The scan history is not separately encrypted; it relies on Android file-based encryption (FBE, on by default on API 26+), which protects data at rest when the device is locked. SQLCipher can be layered in if a stronger threat model requires encrypted database storage, but the current model treats FBE as sufficient for a single-user offline app.

The model asset is protected by the SHA-256 ModelIntegrity guard. An attacker who replaces the .tflite file with a tampered or backdoored model will trigger a digest mismatch, which the result screen surfaces as a warning banner above the diagnosis rather than blocking the scan. The design choice to inform rather than block reflects the operational reality that a grower mid-scan should not be locked out by a guard they did not invoke; the warning makes the tampering visible so the user can choose to distrust the result, while preserving the application's usability. The guard does not prevent tampering—an attacker with write access to the app's private storage can also patch the pinned digest—but it raises the cost of a stealthy model replacement and makes any such replacement observable to the user.

The application was built in eight phases, each ending with a commit, push, and report so progress was never lost. The phases and their deliverables are detailed below.

Phase 1 – Project foundation and navigation skeleton. The Gradle Kotlin DSL build (root and app build.gradle.kts, gradle/libs.versions.toml version catalog, Gradle 8.9 wrapper), the AndroidManifest.xml, the Material 3 light/dark theme with dynamic color, the splash screen, the adaptive launcher icon, the backup/data-extraction rules, the strings and colors, the @HiltAndroidApp LeafGuardApplication, the single-activity MainActivity, and the navigation graph with honest placeholders for every screen. Validation was a JVM unit test for the Routes helpers.

Phase 2 – PIN authentication and session gating. The PinStore over EncryptedSharedPreferences keyed to the Android Keystore (storing a salted PBKDF2-HMAC-SHA256 derived key, never the raw PIN), the PinHasher and PinValidator, the AuthRepository/AuthRepositoryImpl with the StateFlow<AuthState>, the AuthViewModel, the AuthScreen rewired to the view model, the auth-gated LeafGuardNavHost, and

TABLE IX
The 38 PlantVillage classes (canonical index order).

Idx	Class	Idx	Class
0	Apple Scab	19	Pepper healthy
1	Apple Black rot	20	Potato Early blight
2	Apple Cedar rust	21	Potato Late blight
3	Apple healthy	22	Potato healthy
4	Blueberry healthy	23	Raspberry healthy
5	Cherry Powdery mildew	24	Soybean healthy
6	Cherry healthy	25	Squash Powdery mildew
7	Corn Cercospora	26	Strawberry Leaf scorch
8	Corn Common rust	27	Strawberry healthy
9	Corn Northern Leaf Blight	28	Tomato Bacterial spot
10	Corn healthy	29	Tomato Early blight
11	Grape Black rot	30	Tomato Late blight
12	Grape Esca	31	Tomato Leaf mold
13	Grape Leaf blight	32	Tomato Septoria leaf spot
14	Grape healthy	33	Tomato Spider mites
15	Orange Citrus greening	34	Tomato Target spot
16	Peach Bacterial spot	35	Tomato Yellow Leaf Curl
17	Peach healthy	36	Tomato mosaic virus
18	Pepper Bacterial spot	37	Tomato healthy

TABLE X
v3.5 (DINOv2 + DANN) training hyperparameters.

Hyperparameter	Value
Backbone	facebook/dinov2-small (ViT-S/14)
Input resolution	224×224
Feature dimension	384
Total parameters	≈22.1 M
Disease head	Linear(384, 38)
Domain head	Linear(384, 64) → ReLU → Linear(64, 2)
Optimizer	AdamW
Weight decay	0.01
Batch size	32
Phase 1 (linear-probe)	3 epochs, LR 1×10^{-3}, $\lambda_d = 0.1$
Phase 2 (partial unfreeze)	8 epochs, LR 1×10^{-4}, decay 0.85, $\lambda_d = 0.3$
Phase 3 (full unfreeze)	4 epochs, LR 5×10^{-6}, decay 0.9, $\lambda_d = 0.1$
LR scheduler	CosineAnnealingLR (per phase)
CutMix/MixUp probability	0.5 ($\lambda \sim \text{Beta}(0.4, 0.4)$)
Consistency weight λ_c	0.2 (field only, Phase 2)
Field oversampling	3× (WeightedRandomSampler)
Class weighting	Effective-number, $\beta = 0.999$
Test-time augmentation	3 views: orig, hflip, scale 0.875
ONNX opset	17 (dynamo=False)
Deployed model size	86.85 MB (FP32 TFLite)

the Hilt SecurityModule and RepositoryModule. Validation was 38 JVM unit tests with zero failures.

Phase 3 – Room persistence and plants CRUD. The Plant entity, PlantDao with Flow return types for observe and suspend for mutations, LeafGuardDatabase version 1, the PlantRepository/PlantRepositoryImpl, the Hilt DatabaseModule, and the real PlantsScreen and PlantDetailScreen with add-plant and delete-with-confirm. Validation brought the cumulative JVM test count to 50.

Phase 4 – TFLite model and explainable-AI engine. The model assets under assets/models/, the DiseaseClassifier wrapping the TFLite Interpreter (preprocessing to $[1, 224, 224, 3]$ float $[0, 1]$, inference, top-k labels), the SaliencyGenerator producing the occlusion-sensitivity heatmap, the SuggestionEngine and LabelParser/SuggestionParser, the ScanResultHolder singleton, the Hilt InferenceModule, and the real ScanScreen and ResultScreen. Validation brought the cumulative JVM test count to 66 and verified the TFLite conversion off-device against the ONNX reference.

Phase 5 – Live capture. The CameraX PreviewView viewfinder and ImageCapture still capture, wired into the ScanViewModel so a capture runs the classify-saliency-save-persist-navigate flow, with the source bitmap saved as compressed JPEG under filesDir/scans/ and the scan entity persisted with the image path before navigation to result/{scanId}.

Phase 6 – Dashboard, history, records, and images. The DashboardViewModel with live scan and plant lists and summary stats, the HistoryViewModel with the in-memory scan-to-plant join and per-plant filter, the RecordsViewModel with the CsvGenerator (pure RFC 4180) and PdfExporter (Android PdfDocument), and the ImagesViewModel with the Coil AsyncImage gallery grouped by plant.

Phase 7 – Suggestion engine. The Severity and TreatmentStage enums, the production disease_suggestions.json asset authored from integrated-pest-management references, the SuggestionParser reading it via org.json, and the SuggestionEngine exposing suggest and severityFor, with the staged-treatment advisory rendered on the result screen.

Phase 8 – Testing and release. The LeafGuardDatabaseMigrationTest (seeds a real v1 database, opens at v3 with both migrations, asserts the plant survives and

TABLE XI
GitHub release history of the LeafGuard application.

Tag	Theme / assets
v0.2.0	Phase 2: PIN authentication. Debug APK 41.9 MB.
v0.3.0	Phase 3: Room + Plants CRUD. Debug APK 42.0 MB.
v0.4.0	Phase 4: TFLite inference + XAI. Debug APK 50.7 MB; release APK 131.1 MB.
v1.0.0	Production release (all 8 phases). Release APK 131.1 MB.
v1.4.0.0	Swin / PlantVillage backup (pre-JK-TK). Debug APK 149.5 MB; Swin model 8.66 MB.
v1.5.0.0	JK-TK YOLOv11 detector (lab + field). Debug APK 268.4 MB; JK-TK model 217.5 MB.

the scans gain the image-path columns), the JVM helper extraction for the PdfExporter record-line logic, the signed release APK with v1+v2+v3 signing built by CI from a base64-decoded keystore, the apksigner verify CI step, and the v1.0.0 GitHub Release with the signed APK as a raw asset.

Each phase's validation was a green CI build plus a manual device run, and the phased structure is what made the model swap from the Swin asset to the DINOv2 asset a low-risk change late in development: because the I/O contract was fixed in Phase 4 and never depended on the model architecture, swapping the asset touched only the model filename, the SHA digest, and nothing in the application logic.

A. Installation

The signed release APK is published as a raw .apk asset on the repository's GitHub Releases page. To install, a grower enables installation from unknown sources in the Android security settings (the setting's exact name varies by OEM), downloads the APK, and opens it; the v1+v2+v3 signing ensures the APK installs on OEM installers that reject v2-only signatures. The minimum Android version is 8.0 (API 26), and the app targets devices with 6 GB or more of RAM because of the TFLite interpreter's resident footprint. No Google account, no network connection, and no permissions beyond camera and media storage are required; camera permission is requested only when the user first taps the scan screen, and a photo-picker import path is available for users who decline camera permission or prefer to scan an existing photo.

B. First Launch and Registration

On first launch the application forces registration: the grower chooses a 6-digit PIN, confirms it, and enters a display name. The PIN is the only authentication the application will ever perform, so it should be memorable but not trivially guessable; the application derives a salted PBKDF2 key from it and stores only the derived key, so the raw PIN is never on the device. After registration the application drops the user onto the dashboard. On every subsequent launch the application routes to the login screen, where the PIN is entered to unlock; a reset dialog (reachable from the login screen) clears the stored credentials and re-registers, which is the recovery path for a forgotten PIN, at the cost of losing the local scan history because the Room database is keyed to the previous credentials.

C. Scanning a Leaf

To diagnose a leaf, the grower taps the scan destination in the bottom navigation (or the "Scan this plant" action on a plant's detail screen, which prefills the plant association). The scan screen offers a live CameraX viewfinder for a fresh capture and a photo-picker import for an existing image. On capture or import the application center-crops the bitmap to a square, scales it to 224×224, normalizes the pixels to $[0, 1]$, and runs the 3-view test-time-augmented classification; the diagnosis appears on the result screen within roughly half a second. The occlusion heatmap streams onto the result screen over the following 6–32 seconds, produced by the 65-pass occlusion sweep running on an application-scoped coroutine. The result screen shows the predicted condition, a confidence bar, the source image and the heatmap side by side once the heatmap completes, the top-k alternatives, and the staged treatment advisory.

D. Interpreting the Heatmap and the Integrity Banner

The occlusion heatmap shows, in warm colours, the regions of the leaf whose occlusion most reduced the predicted class's confidence—that is, the regions the model relied on. A heatmap that lights up on the lesion is evidence the model attended to the disease evidence; a heatmap that lights up on the background or on a healthy part of the leaf is a warning that the prediction may not be lesion-grounded, and the grower should treat the diagnosis with more scepticism. If the SHA-256 model-integrity check has failed (for example, because the model asset was corrupted or replaced), a warning banner appears above the diagnosis; a diagnosis under a warning banner should be treated as unverified, and the grower should reinstall the application from the official release before acting on it.

E. Records and Sharing

The records screen groups scans by plant into a per-plant disease log and exports it to CSV or PDF on-device. The CSV is RFC 4180-compliant and opens in any spreadsheet application; the PDF is laid out as a paginated per-plant table suitable for handing to an extension officer or keeping in a farm binder. Both exports are generated entirely on-device; no scan data ever leaves the phone. The advisory content in the exports is general integrated-pest-management guidance and should be confirmed against the local extension service before any chemical treatment, because the correct product and its legal application rate vary by jurisdiction, crop stage, and local resistance patterns.

This appendix records the methodology and result of each of the seven Kaggle training notebooks in the development arc, in chronological order from the earliest to the most recent. The notebooks are the primary artifacts behind the field-accuracy trajectory reported in Section VI-F; each entry below distills the notebook's dataset, model, training recipe, augmentation, domain-adaptation approach, loss, test-time augmentation, conversion, reported result, and diagnosed failure.

F. Notebook 1: leafguard-finetune-v1 (mobile baseline)

A TensorFlow/Keras mobile-first baseline built to escape a failing prior Swin model. The earlier v1.4.0.0 Swin model had landed near 36% confidence on diseased tomato and misclassified healthy tomato as strawberry and sweet-potato as tomato on field photographs, attributed to softmax overfitting on PlantVillage's uniform lab backdrop. This notebook used a single PlantVillage source with a stratified subsample capped at 1,200 images per class, a MobileNetV3-Small backbone (≈2.5 M parameters, ImageNet weights, include_preprocessing=False), and a two-stage recipe (4 frozen epochs at Adam 1×10^{-3}, then 6 epochs with the top 30 backbone layers unfrozen at 1×10^{-5}, batch 64, ReduceLROnPlateau and early stopping). Augmentation was a Keras photometric stack (flip, rotation, zoom, translation, brightness, contrast) plus a custom random-erasing with probability 0.5; no CutMix/MixUp, no consistency, no field data, no domain adaptation. Conversion was TF-only: SavedModel → TFLiteConverter with Optimize.DEFAULT and float16, yielding a ≈2.5 MB FP16 TFLite model whose SHA-256 was emitted for the app's integrity check. The notebook established the mobile deployment contract but, having no field data, could not fix the field failures it set out to fix.

G. Notebook 2: leafguard-swin-finetune-v1 (multi-source PV pool)

The first attempt to fine-tune the existing Swin plant-disease model on a merged multi-source PlantVillage pool. Three PlantVillage sources were pooled (the vipoooool augmented set, the abdallahalidev color/segmented/grayscale variants, and the karagwaanntreasure augmented set), totaling 518,000 gathered images—but on inspection all three were derivatives of the same ≈172,000 lab leaves. Deduplication used a cross-source full-file SHA-256 that ran for approximately nine hours and timed out on the T4 runtime. The backbone was the PlantVillage-overfit Swin-Tiny state dict (≈112 MB, 38 labels) from the private dataset, with a two-stage progressive unfreeze (stage 1: freeze encoder layers 0–2,

TABLE XII
Requirements and acceptance matrix.

Requirement	Implementation	Verification
Diagnose 38 PlantVillage classes on-device	DINOv2 v3.5 TFLite graph, 86.8 MB, FP32	Bit-identical ONNX check; field 86.7%
No network dependency for inference	No network permission; model ships in APK	Manifest audit; CI APK build
Field-robust classification	DINOv2 + DANN GRL + consistency + 3× field oversample	Field validation set, 525 images
Explainable predictions	Occlusion-sensitivity 8×8 heatmap, jet ramp	OcclusionRectsTest; device run
Model integrity at runtime	SHA-256 ModelIntegrity singleton, warning banner	PixelNormalizeTest; device run
PIN authentication	PBKDF2-HMAC-SHA256, Keystore-backed	PinHasherTest, EncryptedPinStoreTest
Plant & scan persistence	Room v3 DB, Plant + ScanEntity, migrations	PlantDaoTest, migration test
Per-plant disease history	History join + per-plant filter	HistoryViewModel device run
CSV/PDF records export	Pure RFC 4180 CSV + PdfDocument PDF	JVM helper tests; device PDF run
Image gallery	Coil AsyncImage, source-image grouping	ImagesViewModel device run
Signed release APK	v1+v2+v3 signing, CI keystore	apksigner verify CI step
Reproducible model swap	I/O contract preserved Swin→DINOv2	One-line config + SHA update

train layer 3 + classifier, 2 epochs at 2×10^{-5}; stage 2: unfreeze layer 2, 2 epochs at 1×10^{-5}, batch 32, AdamW, cosine LR). Augmentation was the standard photometric stack plus Gaussian blur and random erasing; no CutMix/MixUp, no field data, no domain adaptation. Export was ONNX opset 17 with the SwinExportWrapper baking ImageNet normalization and softmax into the graph. The base lab accuracy was 98.59%. The version failed to fix the field misclassifications because it had zero field coverage, and the SHA-256 dedup timed out.

H. Notebook 3: leafguard-swin-finetune-v2 (first field data)

The first introduction of real field data and fast dedup. The two duplicate PlantVillage sources were dropped, keeping vipoooool as a small lab anchor and adding three field sources: PlantDoc (≈2.6k field images), the alexzcheny field-acquired set, and the corn field collection. Class-name mapping used an explicit dictionary plus a token-Jaccard fuzzy fallback with a 0.5 threshold, with every mapping logged to an audit JSON. Dedup was replaced with an MD5 of the first 16 KB plus file size, within-source, removing the SHA-256 bottleneck. Caps were declared (LAB_CAP_TRAIN=100, FIELD_CAP=600) but a bug meant the lab cap was never enforced, so PlantVillage (157,000 images) drowned the field data (25,000 images). The training recipe, augmentation, and ONNX export were as in v1. Field accuracy on PlantDoc field photographs collapsed to approximately 11% (89% correct-or-unknown) even as mixed accuracy reached 84.7%. The diagnosed failure—lab drowning field—motivated the next version's separate lab/field capping.

I. Notebook 4: leafguard-swin-finetune-v2-1 (cap enforcement)

The bug-fix release. Lab and field caps were enforced separately (LAB_CAP_TRAIN=100 now applied), and a pure-field validation set was built and reported independently of the mixed validation set. The training recipe, augmentation, and ONNX export were as in v2.0, with the addition of a dynamo=False retry loop in the ONNX export (the v2.0 dynamo default had required the onnxscript package and failed). Capping lab to 100 images per class, however, destroyed the species-discrimination prior (apple-rust began confusing with corn-rust), and only 5 epochs were too few for the 8,000 hard mixed images. The model underfit: both mixed and field accuracy sat at approximately 49%, with loss still falling and validation still rising at the final epoch.

J. Notebook 5: leafguard-swin-finetune-v2-2 (three-stage schedule)

The first three-stage, 15-epoch schedule. Lab was raised back to 400 per class to retain the species-discrimination prior, a CORN_CAP=800 was added to prevent corn dominance (corn has 22,000 images over only 4 classes), and effective-number class weighting ($\beta = 0.999$) replaced inverse-frequency weighting. Label cleanup corrected a grape_leaf folder that had been fuzzy-mis-mapped to a Grape disease class (it is healthy). The three stages were: head-only (3 epochs at 5×10^{-4}), partial unfreeze (8 epochs at 1×10^{-5}, layers 2–3 + pooler + head trainable), and a targeted boost (4 epochs at 3×10^{-6}, with the worst-recall classes oversampled 3×). Model selection switched to field validation accuracy. Augmentation was strengthened (rotation 20, stronger colour jitter, affine shear 15, perspective 0.2). Field accuracy, however, stuck at approximately 34% across all 15 epochs, even as lab climbed from 43% to ≈80%. The failure pattern was cross-species confusion of healthy leaves (grape→apple, peach→blueberry): the model could not identify plant species on field images. This was the documented PlantVillage→PlantDoc ceiling.

K. Notebook 6: leafguard-swin-finetune-v3 (ImageNet pivot)

The pivot. The PlantVillage-overfit base was abandoned for an ImageNet-pretrained Swin-Tiny (microsoft/swin-tiny-patch4-window7-224, ignore_mismatched_sizes=True), and a domain-bridge recipe was added. The dataset plan was reused from v2.2 unchanged. The two-stage schedule was: linear probe (3 epochs at 1×10^{-3}, backbone frozen, classifier only) then full fine-tune (12 epochs at 2×10^{-5}, with layer-wise LR decay of 0.75 across the four encoder stages, cosine LR). CutMix/MixUp with probability 0.5 ($\lambda \sim \text{Beta}(0.4, 0.4)$) and three-fold field oversampling via WeightedRandomSampler were added. Test-time augmentation was introduced: 4 views (original, horizontal flip, $\pm10^\circ$ rotation), softmaxes averaged. The ONNX export preserved the I/O contract so the app needed no changes. Field accuracy jumped to 67.3% and lab to 99.3%, breaking the ≈70 percentage-point PlantVillage→PlantDoc gap. The residual field errors were now same-species disease confusion (potato early versus late blight, corn northern versus cercospora leaf blight).

L. Notebook 7: leafguard-swin-finetune-v3-5-0-0 (DINOv2 + DANN, final)

The final version. The backbone was swapped entirely to self-supervised DINOv2 ViT-S/14 (facebook/dinov2-small, ≈21 M parameters, 384-dimensional CLS-token features) via a custom

LeafGuardDino module with a 38-way linear disease head and a 2-class domain head (Linear→ReLU→Linear). Explicit unsupervised domain adaptation was added: a gradient-reversal layer feeding the domain head, with λ_d scheduled 0.1→0.3→0.1 across the three steps. The three-step progressive unfreeze was: linear probe (3 epochs, LR 1×10^{-3}, backbone frozen, GRL $\alpha = 0.1$), partial unfreeze of the last four ViT blocks (8 epochs, LR 1×10^{-4}, discriminative LR decay 0.85, GRL $\alpha = 0.3$, CutMix/MixUp 0.5, multi-crop consistency regularization weight 0.2 applied to field images only, three-fold field oversampling), and full unfreeze (4 epochs, LR 5×10^{-6}, decay 0.9, GRL $\alpha = 0.1$, consistency dropped). Multi-crop augmentation yielded a strong-aug crop and a weaker consistency crop per image. Test-time augmentation was 3 views (original, flip, 0.875 scale). The export dropped the domain head and baked ImageNet normalization and softmax into the ONNX graph (opset 17, dynamo=False), preserving the v3 I/O contract for a one-line model swap. Field accuracy reached 86.7%, a 19.4 percentage-point gain over the v3 baseline, while lab stayed at 99.3% and mixed rose to 96.1%.

This appendix consolidates the equations governing the v3.5 model for reference. The backbone $f_\theta : \mathbb{R}^{224\times224\times3} \to \mathbb{R}^{384}$ is the DINOv2 ViT-S/14 feature extractor parameterized by θ, producing the CLS-token feature $z = f_\theta(x)$ for an input image x. The disease head is a linear classifier $h_c(z) = \mathrm{softmax}(W_c z + b_c) \in \mathbb{R}^{38}$ with $W_c \in \mathbb{R}^{38\times384}$, trained with the cross-entropy

$$\mathcal{L}_{cls} = -\sum_{k=1}^{38} y_k \log h_c(z)_k, \tag{4}$$

where y is the one-hot disease label. The domain head h_d is connected to the backbone through a gradient-reversal layer R_{λ_d} that is the identity in the forward pass, $R_{\lambda_d}(z) = z$, and negates the gradient in the backward pass by λ_d, $\partial R_{\lambda_d}/\partial z = -\lambda_d I$. The domain loss is the binary cross-entropy of $h_d(R_{\lambda_d}(z))$ against the domain label $d \in \{\text{lab}, \text{field}\}$:

$$\mathcal{L}_{dom} = -\big[d \log h_d(z)_d + (1-d)\log(1-h_d(z)_d)\big]. \tag{5}$$

The consistency regularization penalizes the KL divergence between the strong-aug crop prediction and the consistency-crop prediction, applied only to field images ($\mathbf{1}_{field}$):

$$\mathcal{L}_{cons} = \mathbf{1}_{field}\, D_{KL}\big(h_c(z^{cons}) \,\|\, h_c(z^{strong})\big). \tag{6}$$

The total objective is $\mathcal{L} = \mathcal{L}_{cls} + \lambda_d \mathcal{L}_{dom} + \lambda_c \mathcal{L}_{cons}$ with $\lambda_c = 0.2$ and λ_d following the schedule $0.1 \to 0.3 \to 0.1$ across the three phases. The CutMix/MixUp mixing draws $\lambda \sim \mathrm{Beta}(0.4, 0.4)$, takes $\lambda \leftarrow \max(\lambda, 1-\lambda)$, and replaces the per-sample cross-entropy with the mixed objective $\lambda\mathcal{L}_{cls}(a) + (1-\lambda)\mathcal{L}_{cls}(b)$ for the mixed pair (a, b). The test-time augmentation averages the softmaxes of three views:

$$\bar{p}(x) = \tfrac{1}{3}\big[p(t_{orig}(x)) + p(t_{hflip}(x)) + p(t_{scale_{0.875}}(x))\big], \tag{7}$$

and the occlusion saliency for cell (i, j) of the 8×8 grid is the confidence drop $S_{ij} = p_c(x) - p_c(x \odot \mathbf{1}_{\neg ij})$, smoothed by the separable Gaussian $[1, 2, 1]/4$, soft-normalized by $\sqrt{\cdot}$ against the maximum, and color-mapped through the jet ramp. The effective-number class weight for class k with sample count n_k is $w_k = (1-\beta^{n_k})/(1-\beta)$ with $\beta = 0.999$, which reweights the cross-entropy to reduce the imbalance bias without the over-weighting that naive inverse-frequency produces on rare classes.

M. Training Environment

The seven training notebooks were executed on Kaggle with a single Nvidia Tesla T4 GPU, Python 3, PyTorch 2.9+, the HuggingFace transformers and datasets libraries, onnx and onnxruntime for export and verification, and onnx2tf plus tensorflow for the TFLite conversion step (executed off-notebook for the final asset). The private base-model dataset is kushagradubey09/plant-disease-xai-models (the Swin source checkpoint and DINOv2 reference); the public data sources are vipoooool/new-plant-diseases-dataset (PlantVillage augmented), nirmalsankalana/plantdoc-dataset (PlantDoc field), smaranjitghose/corn-or-maize-leaf-disease-dataset (corn field), and alexzcheny/testdataset (field-acquired). Random seeds are fixed at the start of each notebook for reproducible splits, sampling, and the three seeded bit-identical verification inputs.

N. Re-running a Version

Each notebook is self-contained: it mounts the Kaggle datasets, builds the data manifest, constructs the model, runs the staged training, evaluates on the three validation splits with TTA, exports the ONNX graph, and writes a metrics JSON and a data-manifest JSON as artifacts. Re-running a version reproduces its result up to non-determinism in the GPU kernels and the WeightedRandomSampler draw order, which is why the bit-identical verification is performed against the ONNX reference on seeded inputs rather than against a re-trained checkpoint. The final v3.5 notebook produces the .onnx graph that is then converted to the shipped plant_disease.tflite by the off-notebook onnx2tf/TFLiteConverter pipeline, with the ImageNet normalization and softmax baked in as ONNX nodes before conversion.

O. Application Build

The Android application builds with Gradle 8.9 (Kotlin DSL, version catalog), JDK 17, Android Gradle Plugin 8.x, KSP for Hilt and Room, and a minSdk of 26 and targetSdk matching the current Android release. The committed plant_disease.tflite is an untrained demo fallback that keeps the project buildable without credentials; the CI workflow pulls the trained model from the private Kaggle dataset using a CI secret and overwrites the asset before the release assemble step. The signed release APK is built from a base64-decoded keystore (KEYSTORE_BASE64) with v1+v2+v3 signing enabled and verified by apksigner verify. The full build and test commands are ./gradlew :app:testDebugUnitTest (66 JVM tests), ./gradlew :app:assembleDebug, and ./gradlew :app:assembleRelease.

References

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," Frontiers in Plant Science, vol. 7, p. 1419, 2016.

[2] D. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of computer vision diagnostics," arXiv preprint arXiv:1511.08060, 2015.

[3] S. Murugavalli and R. Gopi, "Plant leaf disease detection using vision transformers for precision agriculture," Scientific Reports, vol. 15, p. 22361, 2025.

[4] S. Nishankar, V. Pavindran, T. Mithuran, S. Nimishan, S. Thuseethan, and Y. Sebastian, "ViT-RoT: Vision transformer-based robust framework for tomato leaf disease recognition," AgriEngineering, vol. 7, no. 6, p. 185, 2025.

[5] Z. Arshad, A. Javed, and A. Saudagar, "ConvGeM-next: A deep learning framework for plant disease detection," Frontiers in Plant Science, vol. 17, p. 1763739, 2026.

[6] M. Sharif, Y. Zhong, M. Sajid, and F. Marinello, "Real-field-ready and digitally sustainable plant disease recognition via federated multimodal edge learning," Agriculture (MDPI), vol. 16, no. 9, p. 918, 2026.

[7] W. Jeon, T. Kim, S. Choi, and K. Yang, "Bridging the lab-to-field gap in plant disease diagnosis through unsupervised domain adaptation," SSRN Electronic Journal, 2025.

[8] B. R. Reddy et al., "Deep learning based mobile application for automated plant leaf disease detection," IEEE, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/11039794

[9] S. Natarajan, P. Chakrabarti, and M. Margala, "Robust diagnosis and meta visualizations of plant diseases through deep neural architecture with explainable AI," Scientific Reports, vol. 14, p. 13695, 2024.

[10] M. Batra and P. Sharma, "Hybrid explainable DeiT-based framework for plant disease classification and severity estimation," Research Square preprint, 2026.

[11] M. Rithiga and R. Nadesh, "SSL VNet: An explainable multimodal hybrid framework integrating self-supervised learning and vision transformers," Frontiers in Artificial Intelligence, vol. 9, p. 1937520, 2026.
[12] M. Oquab et al., "DINOv2: Learning robust visual features without supervision," arXiv preprint arXiv:2304.07193, 2023.
[13] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, and V. Lempitsky, "Domain-adversarial training of neural networks," Journal of Machine Learning Research, vol. 17, pp. 2096–2030, 2016.
[14] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in Proc. Int. Conf. Learn. Representations (ICLR), 2021.
[15] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2021, pp. 10012–10022.
[16] P. Prasad and S. Agniraj, "Cross-domain adaptation techniques for robust plant disease detection: A DANN-CORAL hybrid approach," Int. J. Exp. Res. Rev., vol. 42, pp. 68–84, 2024.
[17] Z. Salman, A. Muhammad, and D. Han, "Plant disease classification in the wild using vision transformers and mixture of experts," Frontiers in Plant Science, vol. 16, p. 1522985, 2025. doi: 10.3389/fpls.2025.1522985.
[18] C. Bai, L. Zhang, L. Gao, L. Peng, P. Li, and L. Yang, "DINOV2-FCS: a model for fruit leaf disease classification and severity prediction," Frontiers in Plant Science, vol. 15, p. 1475282, 2024. doi: 10.3389/fpls.2024.1475282.
[19] A. Islam, T. Tahsin, Z. Anjum, M. B. Hasan, and M. H. Kabir, "A domain-adapted lightweight ensemble for resource-efficient few-shot plant disease classification," arXiv preprint arXiv:2512.13428, 2025.
[20] S. M. Alhammad, D. S. Khafaga, W. M. El-hady, F. M. Samy, and K. M. Hosny, "Deep learning and explainable AI for classification of potato leaf diseases," Frontiers in Artificial Intelligence, vol. 7, p. 1449329, 2024. doi: 10.3389/frai.2024.1449329.